# GUESS

The Graph Exploration System — for the web

Eytan Adar

Manual version: November 2026

## Contents

**1. GUESS features** **3**

**2. Getting started** **5**
2.1 Opening GUESS . . . 5
2.2 The window . . . 5
2.3 Finding help . . . 6

**3. Tutorial** **8**

**4. Getting your data in** **11**
4.1 Opening files . . . 11
4.2 The GUESS .gdf format . . . 11
4.3 Other formats . . . 13
4.4 Data from the web . . . 13
4.5 Building graphs in code . . . 14

**5. Manipulating and querying nodes and edges** **15**
5.1 Nodes, edges and fields . . . 15
5.2 Queries and sets . . . 15
5.3 Visibility . . . 17
5.4 The selection . . . 18
5.5 The data table . . . 18
5.6 SQL . . . 18

**6. Laying out graphs** **20**

**7. Analysis commands** **23**
7.1 Clustering and communities . . . 23
7.2 Visualizing fields . . . 23
7.3 Field, graph, node and edge statistics . . . 25
7.4 Paths and filters . . . 26
7.5 Charts . . . 27
7.6 Random graph generation . . . 28
7.7 Structure, mixing and comparing partitions . . . 28

7.8 networkx from the console . . . 29
7.9 igraph: faster statistics and more methods . . . 29

**8. Modifying graphs** **31**

**9. Output commands** **32**

**10. Several graphs** **33**

**11. States and animations** **34**
11.1 State-sensitive queries . . . 34
11.2 Ranges and time . . . 34

**12. Legends, hulls and the display** **36**
12.1 Legends . . . 36
12.2 Convex hulls . . . 36
12.3 The display . . . 37

**13. Direct manipulation** **39**
13.1 Looking . . . 39
13.2 Selecting . . . 39
13.3 Dragging with meaning . . . 40
13.4 The click tools . . . 41
13.5 Responding to clicks in code . . . 41

**14. The console, history and undo** **43**

**15. Modifying the interface / expanding GUESS** **44**
15.1 Example 1: a simple button . . . 44
15.2 Example 2: a threshold slider . . . 44
15.3 Example 3: a network monitor . . . 45
15.4 Example 4: an app . . . 45
15.5 Code bits . . . 46

**16. Sharing and deploying** **47**
16.1 Sessions . . . 47
16.2 Links . . . 47
16.3 Notebooks and reports . . . 47
16.4 Deploying GUESS . . . 47

**Appendix A. Colors** **48**

**Appendix B. Gython and Python 3** **48**

**Appendix C. Changes from the 2006 version** **49**

**Appendix D. Command reference** **50**
Graph structure, loading and saving, mapping, view . . . 50
Undo, redo, history and logging . . . 55
Statistics, communities, filters and paths . . . 56

Clustering and generators . . . 58
Structure: bridges, cut nodes, backbones, directed graphs, mixing, comparing partitions . 59
networkx bridge and sample graphs . . . 61
igraph: communities, large-graph layouts, similarity, patterns, cuts and flows . . . 61
Layouts . . . 65
Display: hulls, legends, background, edges, labels, selection, export . . . 69
Styles and visual mappings . . . 72
Pie nodes, borders and edge bundling . . . 73
Grouping: collapse and expand . . . 73
Several graphs: subgraphs and tabs . . . 74
Time: ranges, years and the timeline . . . 74
Presentations (slides) . . . 74
Data from the web: URLs, SPARQL, NDEx, Neo4j, packages . . . 75
States and morphing . . . 76
Ranges (existence intervals) . . . 77
Charts . . . 77
Interface building (panels, widgets, timers, events) . . . 77
Sessions and apps . . . 78
SQL . . . 79
Colors . . . 79
Examples . . . 79
Fields . . . 85
Gython syntax . . . 86

# 1. GUESS features

GUESS is a tool for exploring graphs: social networks, citation networks, communication logs, biological pathways, anything with nodes and edges. You look at the graph, and you talk to it in a language built for the job. That was the idea in 2006 and it hasn't changed. What has changed is everything underneath: GUESS now runs in a web browser, there is nothing to install, and the language is Python 3.

This tool is/includes:

- **Gython**, Python 3 extended with operators for selecting and changing nodes and edges. `(freq > 100).color = red` colours every edge whose `freq` field is over 100. `(dept == 'dept4') - (dept == 'dept9')` gives the edges between two departments. These are real Python expressions, so you can store them in variables, loop over them and write functions around them.
- **A zoomable view** of the graph. You can pan, zoom, hover for details, and select with clicks, boxes and lassos. Drags can mean something: after a circle layout, dragging a node reorders the ring.
- **A console made of cells.** Every command you run stays on the screen and can be edited, re-run, undone and redone. Output that mentions nodes or edges is linked back to the graph.
- **Fields and queries.** Nodes and edges carry properties (fields). You query them, map them to colour, size, shape and labels, chart them, and run SQL over them.
- **Analysis**: centrality measures computed on demand, communities (Louvain, edge betweenness, bicomponents), shortest paths, filters, and random graph generators.

- **Layouts**: ForceAtlas2, GEM, Fruchterman–Reingold, Kamada–Kawai, MDS, Sugiyama (layered), radial, circle, tree, hive plots, scatter plots by field, and overlap removal.
- **States and animation.** Save the graph as a named state, change things, and *morph* smoothly between states. Ranges and a timeline let you look at graphs that change over time.
- **Many formats in and out**: GDF (the GUESS format), GEXF (Gephi), GraphML, GML, Graphviz DOT, Pajek, UCINET DL, CSV, URL*s*, SPARQL endpoints (e.g. Wikidata), NDEx and Neo4j. Out: GDF, GEXF, GraphML, GML, CSV, PNG, JPG, GIF, SVG, PDF, PostScript/EPS, WebM movies, notebooks and HTML reports.
- **Interface building.** Panels, buttons, sliders and timers are a few lines of Gython. Package a graph and its interface as an *app*: a link that opens straight into your interactive figure.
- **Help everywhere**: a searchable command and example index, signature hints as you type, a guide to every mouse gesture, and error messages that point at the line and suggest the name you probably meant.

This tool isn't:

- A replacement for Gephi, Cytoscape, UCINET, Pajek or networkx. GUESS is for *exploring*: looking, asking, changing, and looking again. It does a fair amount of analysis, and when you need more it's still Python.
- A tool for enormous graphs. Tens of thousands of nodes are fine. Millions are not what it's built for.

# 2. Getting started

## 2.1 Opening GUESS

GUESS is a web page. Open its address in a current browser (Chrome, Edge, Firefox or Safari) and wait a few seconds. The first visit downloads the Python runtime, which takes a little while. After that it's cached, and GUESS also works offline. When it's ready, the sample graph appears and the console says `loaded 402 nodes, 725 edges`.

Nothing you load leaves your computer. Files you open are read by the page itself, and your work is autosaved in the browser. If you close the tab and come back, the graph, the history and the display are where you left them. Python variables you defined are not kept, but the commands that made them are in the history.

If you want to run your own copy, on a laptop with no network or on your group's web server, see Deploying GUESS.

## 2.2 The window

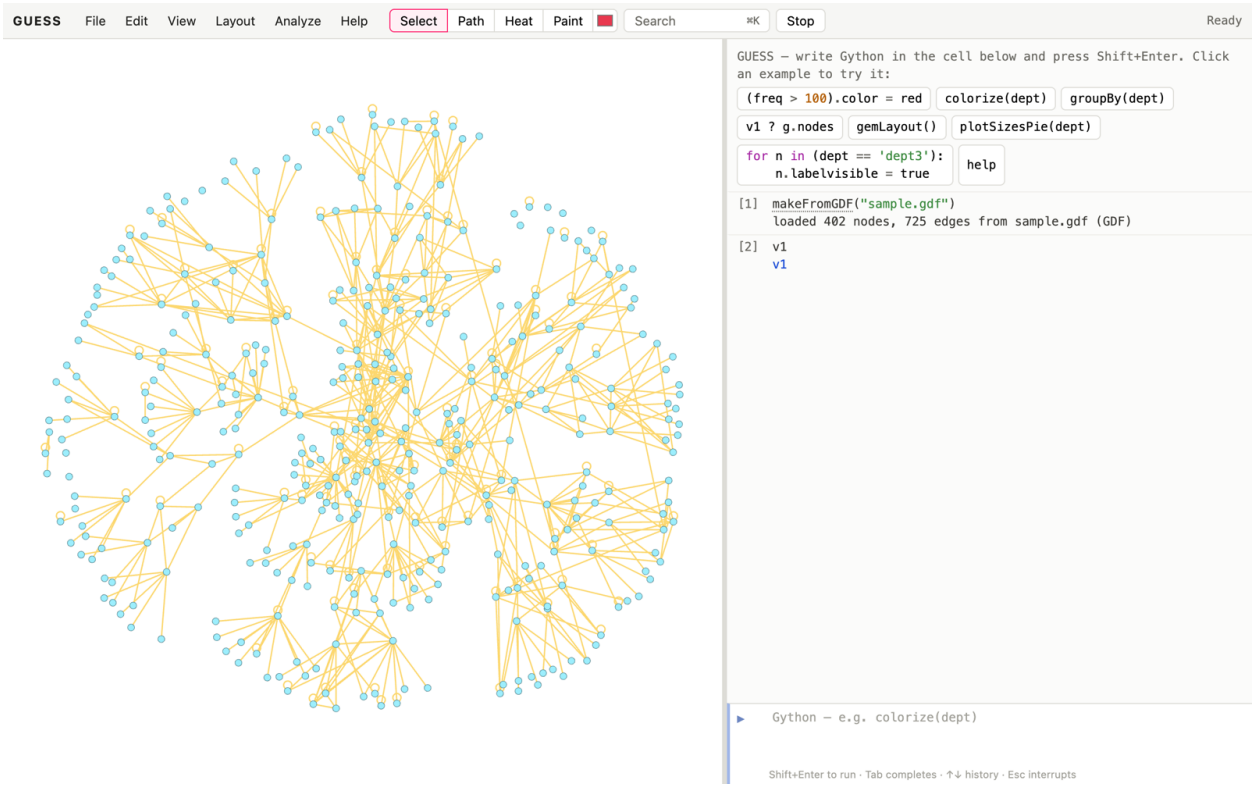



**Figure 1:** The GUESS window. The graph is on the left and the console on the right. The menus and tools are along the top.

The window has three parts:

- **The graph** (left). This is where you look, hover, click and drag.
- **The console** (right). Commands go in the working cell at the bottom. Each command you run becomes a numbered cell above it, followed by its output. You can drag the divider between

the graph and the console. On a narrow screen the console moves below the graph.
- **The menu bar** (top): *File*, *Edit*, *View*, *Layout*, *Analyze* and *Help*, then the click tools (*Select*, *Path*, *Heat*, *Paint* and a paint colour), a *Search* box, *Stop*, and a status line.

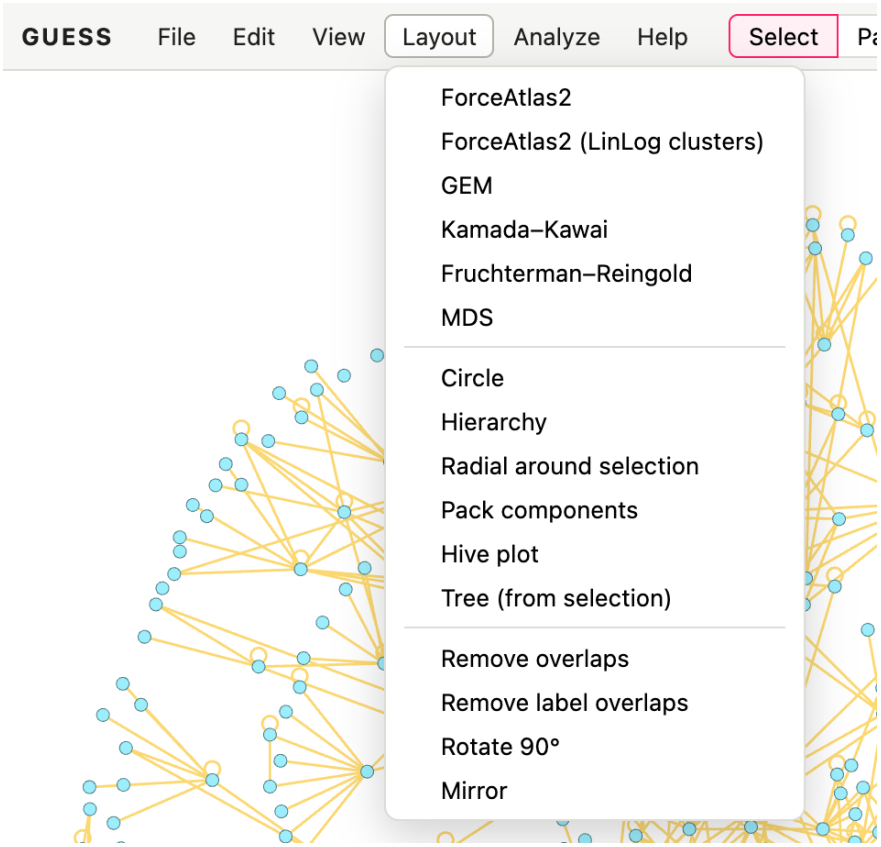



**Figure 2:** The menus collect the commands you'd otherwise type. Every menu item runs a command, and the command shows up in the console, so you can learn the language by using the menus.

Most menu items simply run a Gython command and show it in the console. This is deliberate. If you lay out the graph from the *Layout* menu, the console shows `forceAtlas2Layout()`, and the next time you can type it, change its options, or put it in a script.

## 2.3 Finding help

There are several ways in:

- **Help → Help and examples** (or **F1**) opens a panel with two tabs. *Commands & examples* is a searchable list of every command and a set of worked examples; click an example to put it in the console. *Mouse & gestures* explains, with pictures, everything the mouse does.
- **Search** (or **Cmd/Ctrl+K**) is a command palette. Type a few letters to find commands, examples, menu actions, nodes and fields.
- `help(colorize)` in the console prints the documentation of a command. `help()` alone opens the help panel.
- Hovering over a command name in the console shows its documentation. Typing `colorize(` shows its signature, and **Tab** completes names.

- When something goes wrong, the error names the line and, for misspelled names, the name you probably meant. Click *details* for the full Python traceback.
- **Help** → **Manual** opens this manual in a new tab, as a web page, PDF, EPUB or Word file. **Help** → **GitHub** opens the project's page, with the source code and issue tracker.

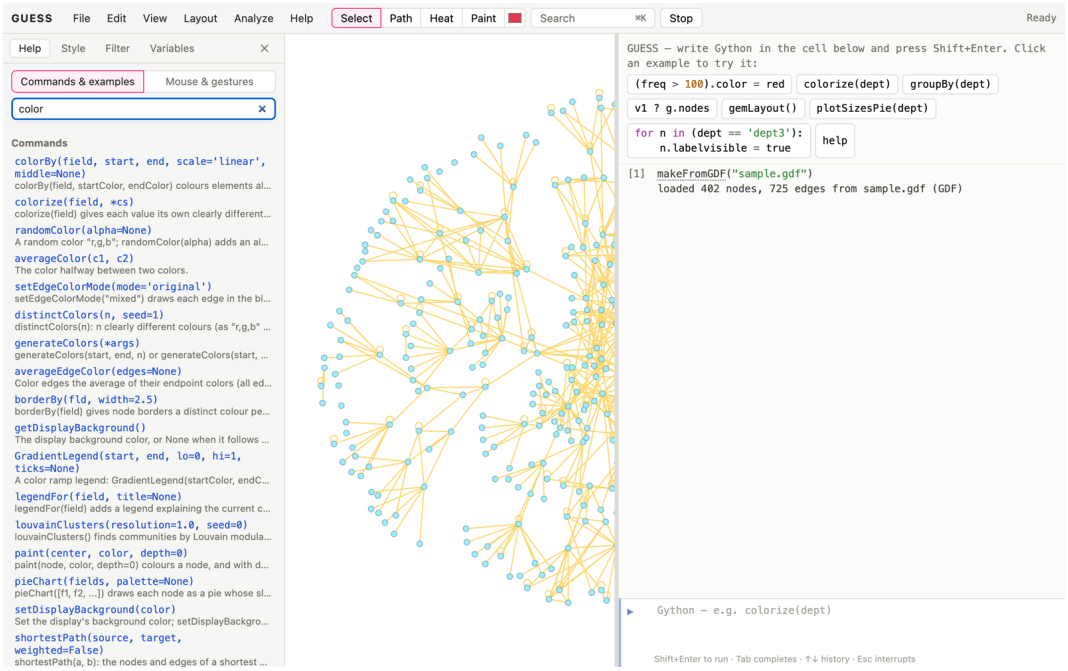



**Figure 3:** The help panel, searching for "color". Commands come first, then examples.

# 3. Tutorial

Let's start with a simple example. The sample graph that GUESS opens with (`sample.gdf`) has about 400 nodes and 700 edges. It represents a corporate communication network. Each node is an employee with a department (the `dept` field). Each edge is communication between two employees, and its `freq` field is the number of messages. If you've replaced the sample with something else, *File* → *Sample graph* brings it back.

First, some basics. Try moving around in this space:

- Drag the background to pan. Scroll or pinch to zoom. Double-click the background to fit the whole graph.
- Hover over a node or edge to see its fields. Hovering an edge also rings its two endpoints, so you can see where it goes.
- Click a node to focus on it and its neighbours. Click again, or press Esc, to clear. Double-click a node to zoom to it.
- Drag a node to move it.

Now click in the working cell at the bottom of the console. You're talking to a Python 3 interpreter. Type 2+2 and press **Shift+Enter** (or Cmd/Ctrl+Enter). It answers 4. Plain **Enter** adds a new line, so longer code is easy to write:

```
def fact(a):
    if a == 1:
        return 1
    else:
        return a * fact(a - 1)

fact(5)
```

Press Shift+Enter and you get 120. Through the console you can also control what you see. Type `center` and press Shift+Enter: the display zooms to fit all the nodes. Commands that take no arguments don't need the parentheses.

Nodes can be selected by name or by a query on their properties. For example, try `(v5, v6).color = red`. Nodes 5 and 6 turn red. Every node is a variable named after it, and assigning to a field of a group of nodes assigns it to each one. Nodes in the sample have a department, so

```
(dept == 'dept5').color = black
```

makes everyone in department 5 black. `dept == 'dept5'` is a query: it returns the set of nodes whose `dept` is `dept5`. Try typing just `dept == 'dept5'`. The console lists the nodes. Hover over one in the output and it lights up in the graph; click it and the view zooms to it.

Edges are accessed in a slightly different way. Edges are named by their end points. `(v67 - v76).color = red` changes the edge between person 67 and 76 to red. You can also find edges by query. Edges here have a `freq` field, so if we wanted to hide edges where the communication frequency was under 100 we would type:

```
(freq < 100).visible = false
```

The `-` operator means an edge in either direction. If the graph had directed edges (this one doesn't) you could also write `v67 -> v76` and `v67 <- v76`. `v67 <-> v76` means an undirected edge, and `v67 ? v76` means any edge at all.

The last way to get at edges is through node sets. Let's say we only care about the communication between departments 4 and 9.

1. Let's hide everything: `g.nodes.visible = false`.
2. Then show only the nodes in departments 4 or 9: `((dept == 'dept4') | (dept == 'dept9')).visible = true`.
3. Colour the edges between the two departments: `((dept == 'dept4') - (dept == 'dept9')).color = red`. This finds all the nodes in department 4 and all the nodes in department 9, then all the edges between them (in this case only one).
4. Colour the edges inside department 9 blue: `((dept == 'dept9') - (dept == 'dept9')).color = blue`.

Because the language is Python, we could have made this simpler with some intermediate variables. The following gives the same result:

```
g.nodes.visible = false
dept4 = (dept == 'dept4')
dept9 = (dept == 'dept9')
(dept4, dept9).visible = true
(dept4 - dept9).color = red
(dept9 - dept9).color = blue
center()
```

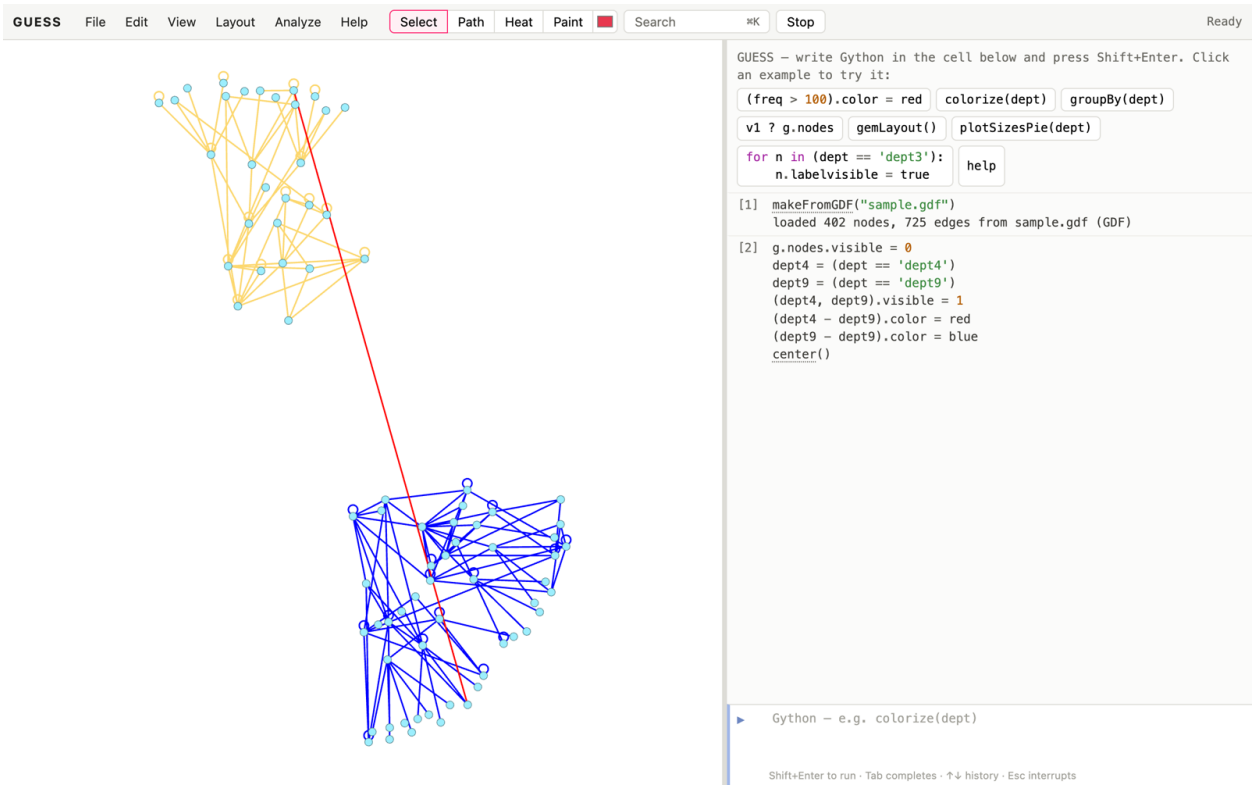



**Figure 4:** Departments 4 and 9. The one edge between them is red, and the edges inside department 9 are blue.

Made a mess? Press **Cmd/Ctrl+Z** (when the cursor isn't in the code cell), choose *Edit* → *Undo*, or

type `undo()`. Each command is one step, and undone cells are struck through in the console. Redo brings them back.

GUESS also has analysis commands that simplify common tasks. First, let's put things back: `g.nodes.visible = true`, `g.edges.color = dandelion` and `g.nodes.color = cornflowerblue` (the defaults). Now type `density()`. This calculates the density of the graph (0.009).

Other commands do more interesting things. `colorize(dept)` gives each department its own colour, chosen to be as different from each other as possible. `colorBy(freq, blue, red)` colours the edges along a range from blue to red by their frequency.

Let's try something a little more interesting:

```
colorize(dept)
deptg = groupBy(dept)
xy = Legend()
for d in deptg: xy.add(d[0], d[0].dept)
```

`groupBy(dept)` returns a set of sets, one for each department. `Legend()` makes an empty legend. The last line goes through the departments and adds a "prototype" node from each, the first one, to the legend, along with its department's name. (The one-liner `legendFor(dept)` does the same thing.)

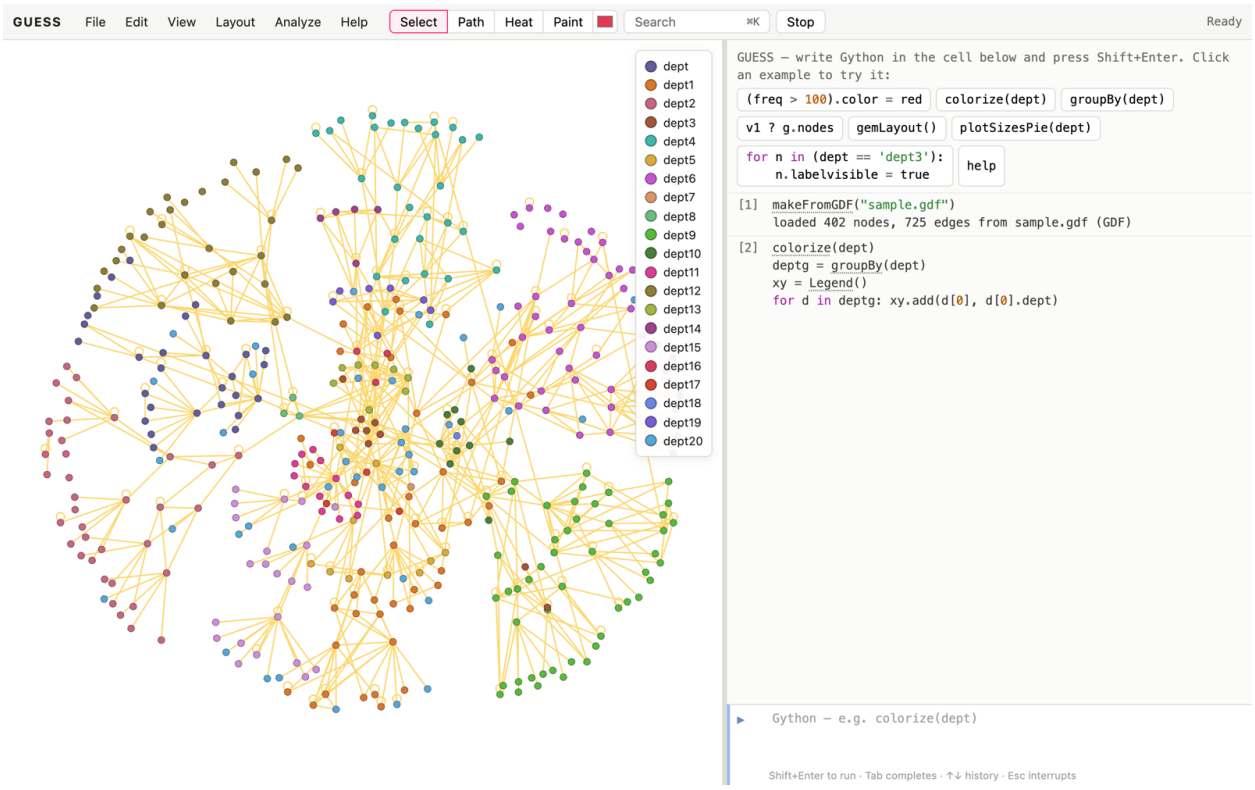



**Figure 5:** colorize(dept) and a legend built by hand.

This is standard Python, and any Python tutorial will help you with the general language. The rest of this manual is about the parts that are particular to GUESS.

# 4. Getting your data in

There are three ways to get a graph into GUESS:

1. **Open a file.** Use *File* → *Open…*, or drag a file onto the window.
2. **Fetch it from the web.** Load a URL, query a SPARQL endpoint, or load an NDEx network or a Neo4j query.
3. **Build it in code.** Use `addNode`, `addEdge` and the random graph generators.

## 4.1 Opening files

*File* → *Open…* (or drag and drop) understands these formats:

| extension | format | command |
|---|---|---|
| `.gdf` | GUESS data file (section 4.2) | `makeFromGDF("f.gdf")` |
| `.gexf` | Gephi | `makeFromGEXF("f.gexf")` |
| `.graphml`, `.xml` | GraphML | `makeFromGraphML("f.graphml")` |
| `.gml` | GML (networkx, igraph, yEd, Cytoscape) | `makeFromGML("f.gml")` |
| `.dot`, `.gv` | Graphviz DOT | `makeFromDOT("f.dot")` |
| `.net` | Pajek | `makeFromPajek("f.net")` |
| `.dl` | UCINET DL | `makeFromDL("f.dl")` |
| `.csv`, `.tsv` | edge table, node table, adjacency matrix or list (as Gephi reads them) | `makeFromCSV("edges.csv", "nodes.csv")` |
| `.guess` | a saved GUESS session | `makeFromSession("s.guess")` |
| `.py` | a Gython script (it runs) | `execfile("s.py")` |
| `.png`, `.jpg`, … | an image, for a background or node images | `setBackgroundImage("map.png")` |

Opening a graph replaces the current one completely, its fields included, so nothing of the old graph lingers (`undo()` brings it back; to keep both, open the new one in another tab, section 10). Computed fields you were using, such as `pagerank`, are computed again for the new graph.

An opened file is also copied into GUESS's own file space, so the command form works on it afterwards. If you dropped `office.gdf` on the window, `makeFromGDF("office.gdf")` reloads it. *File* → *Paste data…* takes a table copied from a spreadsheet.

GML and DOT files bring their layout with them when they have one: GML `graphics [ x y w h fill type ]` and Graphviz `pos`, `width`, `height`, `color`/`fillcolor` and `shape` become positions, sizes, colours and shapes. Every other attribute becomes a field. Nodes in a Graphviz `cluster_*` subgraph get a `cluster` field, so `colorize(cluster)` shows the clusters. `makeFromGML` still opens GraphML too, as it did in GUESS 2006.

Loading a graph replaces the current one. To keep several graphs at once, see section 10.

## 4.2 The GUESS .gdf format

The GDF format is very simple. First we define the nodes and their properties, then the edges and theirs. The node section starts with a line like:

```
nodedef> name
```

The `nodedef>` line tells GUESS what is on each of the following lines. In the simplest case there is one column, the node's name. Names must be unique. GUESS makes a variable for every node, so names that are valid Python names (`v1`, `alice`, `gene_42`) are the most convenient. Other names still work, but you then reach the node with `g.getNodeByName("name with spaces")` instead.

The edge section starts with:

```
edgedef> node1,node2
```

and each line after it names the two endpoints. Here is a complete file:

```
nodedef> name
alice
bob
carol
edgedef> node1,node2
alice,bob
bob,carol
```

Now let's add some properties. Each column after the first is a field: a name, optionally followed by a type and a default value.

```
nodedef> name,dept VARCHAR(32),salary INT default 0,manager BOOLEAN
alice,sales,52000,true
bob,sales,48000,false
carol,engineering,,false
edgedef> node1,node2,freq INT,directed BOOLEAN
alice,bob,12,false
bob,carol,3,true
```

The types are `VARCHAR` (text, the default if you don't say), `INT`/`INTEGER` (also `TINYINT`, `SMALLINT`, `BIGINT`), `DOUBLE` (also `FLOAT`, `REAL`, `DECIMAL`) and `BOOLEAN` (also `BIT`). An empty value takes the default. Values containing commas go in single or double quotes.

Some field names are special. They control how nodes and edges look and behave, and you can set them in the file:

| node field | meaning | default |
|---|---|---|
| `x`, `y` | position (if missing, GUESS lays out the graph) | |
| `color` | fill colour (a name or `"r,g,b"` / `"r,g,b,a"`) | cornflowerblue |
| `strokecolor` | outline colour | darker fill |
| `style` | shape: 1 rectangle, 2 ellipse, 3 rounded rectangle, 4–6 the same with the label inside, 7 image | 2 |
| `width`, `height` | size | 10 |
| `label`, `labelvisible`, `labelcolor` | the label text, whether it's shown, its colour | name, false |
| `image` | an image file or URL (with style 7) | |
| `visible` | whether it's drawn | true |
| `fixed` | layouts don't move it | false |

| edge field | meaning | default |
|---|---|---|
| `directed` | a directed edge (arrow) | false |
| `color`, `width` | colour and thickness | dandelion, 2 |
| `weight` | used by weighted layouts and paths | 1 |
| `label`, `labelvisible`, `labelcolor` | as for nodes | |
| `visible` | whether it's drawn | true |

GDF files embedded in old GUESS applets end each line with `;` and are sometimes written on one line. Both forms load.

### 4.3 Other formats

- **GEXF** (Gephi's format) keeps node attributes, edge attributes, colours, sizes and positions. `exportGEXF("g.gexf")` writes it back for Gephi.
- **GraphML** keeps typed attributes (`<key>`) with their defaults.
- **Pajek `.net`** reads vertices (with their coordinates, shapes and colours), arcs (directed) and edges (undirected), with weights.
- **UCINET DL** reads the edgelist and full-matrix forms, with labels.
- **GML** (networkx, igraph, yEd, Cytoscape) and **Graphviz DOT** keep their attributes and any layout they carry (section 4.1). `exportGML("g.gml")` and `exportDOT("g.dot")` write them. `exportDOT` groups nodes into Graphviz clusters by a field (`exportDOT("g.dot", clusters="modularity_class")`), and `layout=False` leaves positions out so Graphviz lays the graph out itself.
- **CSV** works as in Gephi. An edge table has `Source`, `Target`, and optionally `Type` (directed/undirected), `Weight` and `Label`; other columns become edge fields. A node table has `Id`, optionally `Label`, and other columns. GUESS also recognises adjacency matrices and adjacency lists. Opening only a node table adds its columns to the current graph's nodes.

### 4.4 Data from the web

```
makeFromURL("https://example.org/network.gexf")         # any supported format
makeFromSPARQL("""
  SELECT ?item ?itemLabel ?linked ?linkedLabel WHERE {
    ?item wdt:P31 wd:Q5; wdt:P737 ?linked .               # people and who influenced them
    SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
  } LIMIT 300""")
makeFromNDEx("<network uuid>")                          # a network from ndexbio.org
makeFromCypher("MATCH (a)-[r]->(b) RETURN a, r, b LIMIT 500",
               url="http://localhost:7474", user="neo4j", password="...")
```

`makeFromSPARQL` turns each result row into an edge from the first column to the second (Wikidata is the default endpoint). Columns ending in `Label` name the nodes, and the remaining columns become fields. `sparql(query)` returns the rows without building a graph. A server must allow requests from web pages (CORS) for the browser to reach it; Wikidata and NDEx do.

### 4.5 Building graphs in code

```
newGraph("scratch")              # an empty graph in a new tab (section 10)
addNode("ann", color=red)
addNode("bo")
addEdge(ann, bo, weight=3)
addDirectedEdge(bo, ann)
addNodeField("score", DOUBLE, 0.0)
ann.score = 4.5
```

`addNode` makes a node and a variable with its name. `addEdge` (or `addUndirectedEdge`) and `addDirectedEdge` connect two nodes. Keyword arguments set fields. A new field must be declared with `addNodeField` or `addEdgeField` (giving the type and default) before you assign to it. GUESS won't make a new field from a typo.

Classic networks from the literature load with `makeSample(name)`: `karate` (Zachary's karate club, with each member's `club`), `lesmis` (Les Misérables co-appearances), `florentine`, `davis` (Southern Women, two-mode), `petersen`, `kite`, `tutte`, `dodecahedron` and `caveman`. `makeSample()` lists them.

There are also random graph generators: `makeSimpleRandom(nodes, edges)`, `makeErdosRenyiRandom(n, p)`, `makeBarabasiAlbertRandom(start, add, steps)`, `makeKleinbergRandom(side, exponent)`, `makeLattice1DRandom(n, toroidal)`, `makeLattice2DRandom(side, toroidal)`, `makeEppsteinRandom(n, e, r)` and `makeSimpleDirectedRandom(n, e)`. Each one replaces the current graph.

# 5. Manipulating and querying nodes and edges

## 5.1 Nodes, edges and fields

Every node is a variable named after it: `v1`, `alice`. The variable `g` is the graph. `g.nodes` and `g.edges` are all the nodes and all the edges. Edges are reached through their endpoints: `v1 - v2`, or `v1 -> v2` if direction matters.

A **field** is a property that every node (or every edge) has. Fields are named like variables. `dept` on its own is the department field, and `v1.dept` is v1's department. Fields come from the file you loaded, from the built-in set (section 4.2), from commands that add them (`louvainClusters()` adds `modularity_class`), and from `addNodeField`/`addEdgeField`. The node and edge fields are listed by `getNodeSchema()` and `getEdgeSchema()`. The *View* → *Data table* shows them all.

Fields know a little about themselves:

```
freq.min, freq.max, freq.avg, freq.sum
dept.distinct                   # every value, once each
sortBy(freq)[-5:]               # the five edges with the highest freq
groupBy(dept)                   # a set of sets, one per value
groupAndSortBy(dept)            # the same, largest group first
```

Some node fields aren't stored but computed the first time you use them: `indegree`, `outdegree` and `totaldegree`, and the network statistics in section 7.3, such as `pagerank` and `betweenness`.

Names you've defined are yours, but GUESS protects its own names. You can't overwrite `g`, `dept`, `colorize` or a node's name by accident. Assigning to one gives an error that says so.

## 5.2 Queries and sets

A **query** compares a field with a value and returns the set of matching nodes or edges:

```
dept == 'dept3'                 # nodes
freq > 100                      # edges
totaldegree >= 10
name like 'v1%'                 # SQL-style pattern: % any text, _ one character
dept != 'dept3'
freq > freq.avg                 # compare with a statistic
```

Sets combine with `&` (intersection) and `|` (union), and with `.difference(...)`:

```
(dept == 'dept1') & (totaldegree > 3)
(dept == 'dept1') | (dept == 'dept2')
g.nodes.difference(dept == 'dept1')
```

A tuple or list of nodes also works as a set: `(v1, v2, v3).color = green`.

**Broadcasting.** Setting a field on a set sets it on every member. Reading a field from a set gives a list of values:

```
(dept == 'dept3').color = orange
(dept == 'dept3').size = 20          # size sets width and height together
(dept == 'dept3').labelvisible = true
(dept == 'dept3').totaldegree        # a list, one value per node
```

**Edges between sets.** The edge operators work between nodes and between sets of nodes:

| expression | gives |
|---|---|
| `a -> b` | edges directed from a to b |
| `a <- b` | edges directed from b to a |
| `a <-> b`, `a - b` | edges between a and b (undirected, or either direction) |
| `a ? b` | any edges between a and b |

So `v1 ? g.nodes` is every edge touching v1, and `(dept == 'dept1') - (dept == 'dept3')` is every edge between the two departments.

**From edges to nodes and back.** `.nodes` on a set of edges gives their endpoints. `.edges` on a set of nodes gives the edges touching them:

```
(freq > 100).nodes.labelvisible = true     # label everyone in a busy conversation
(dept == 'dept3').edges.color = gray       # every edge touching department 3
v1.getNeighbors()                          # v1's neighbours
```

(`findNodes(set)` and `findEdges(set)` pick out just the nodes, or just the edges, from a mixed set.)

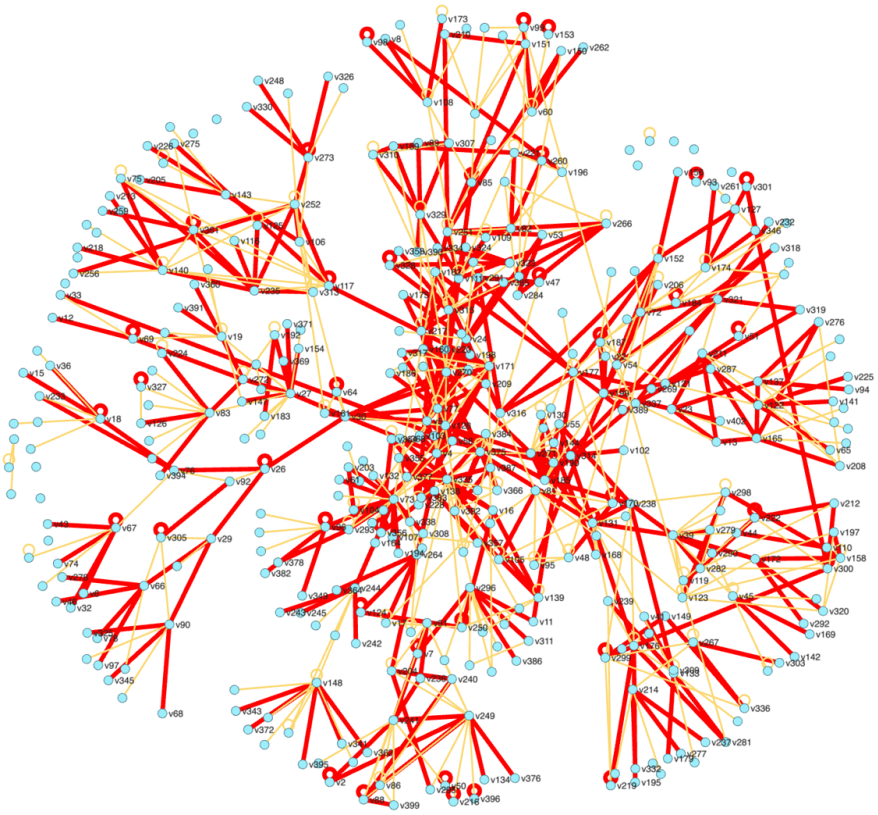


**Figure 6:** (freq > 100).color = red; (freq > 100).width = 5; (freq > 100).nodes.labelvisible = true

Sets are Python sequences. You can loop over them, take `len(...)`, index them and slice them:

```
for n in (dept == 'dept3'):
    n.label = n.name + " (" + n.dept + ")"
len(freq > 100)
```

## 5.3 Visibility

Hidden nodes and edges stay in the graph, so they still show up in queries. Two rules keep the picture consistent. Hiding a node hides its edges, and showing an edge shows its endpoints:

```
(dept == 'dept3').visible = false       # their edges disappear too
(freq > 150).visible = true             # these edges and their endpoints reappear
```

`showOnly(set)` hides everything except the set (nodes, edges, or both), and `showAll()` shows everything again. To take nodes out of the graph rather than hide them, see section 8.

## 5.4 The selection

What you select with the mouse (section 13) is the variable `sel` in the console, and `select(...)` selects from code:

```
select(dept == 'dept2')
sel.color = orange
center(sel)
select()                          # clear it
```

## 5.5 The data table

*View* → *Data table* (or `table()`) opens a spreadsheet of the nodes or edges below the graph. `table(dept == 'dept3')` shows just those. You can sort by a column, filter, and double-click a cell to edit it. An edit is an ordinary command and can be undone. Selecting rows selects the nodes, and vice versa. `InfoWindow.details(v1)` (from the 2006 version) opens the table on one element.

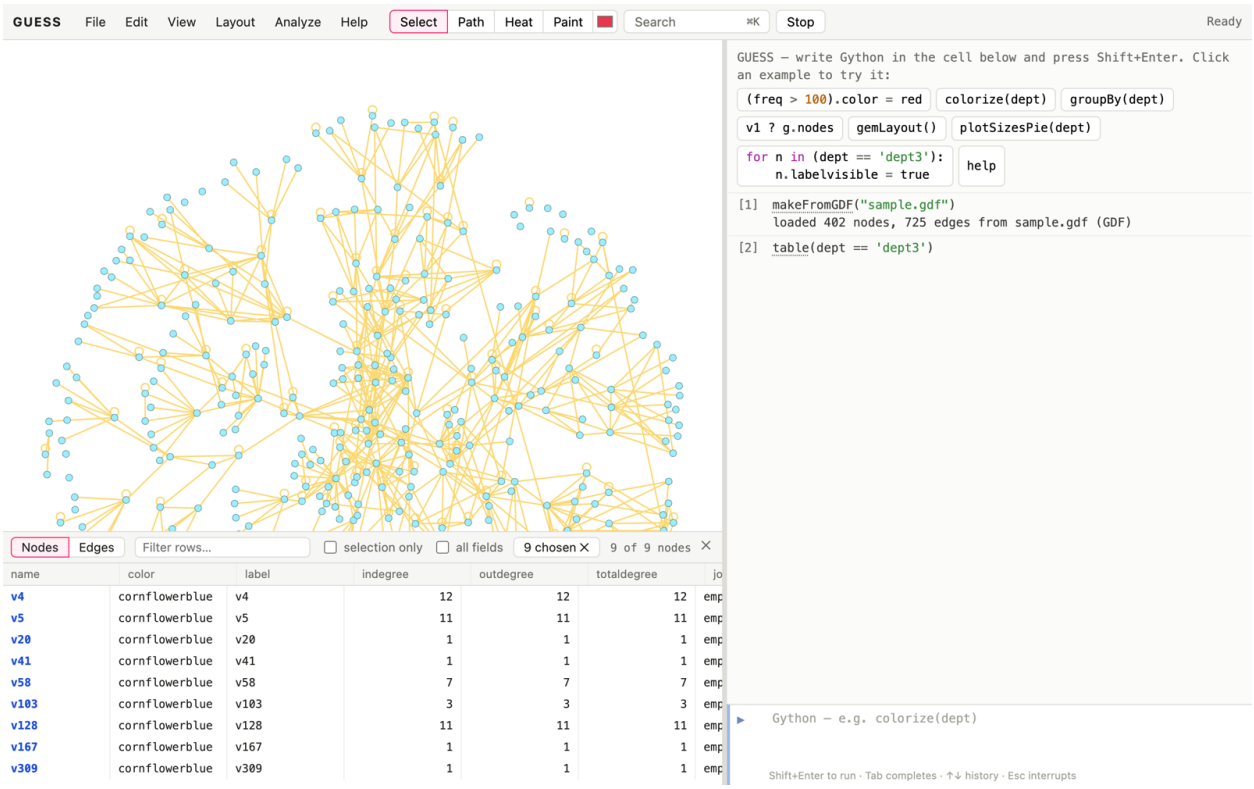



**Figure 7:** table(dept == 'dept3'): the data table below the graph.

## 5.6 SQL

If you'd rather ask in SQL, `sql(...)` puts the nodes and edges in two tables and runs your query:

```
sql("SELECT dept, count(*) FROM nodes GROUP BY dept ORDER BY 2 DESC")
sql("SELECT name FROM nodes WHERE totaldegree > 10")      # a set of nodes
sql("SELECT __edgeid FROM edges WHERE freq > 100")        # a set of edges
sql("UPDATE nodes SET color = 'red' WHERE dept = 'dept3'")
```

A query that returns only `name` (nodes) or `__edgeid` (edges) gives a set, so it combines with the rest of Gython. Other queries give rows. `UPDATE`s change the graph. Inserting and deleting rows isn't supported (use `addNode` and `remove`).

## 6. Laying out graphs

A layout command moves the nodes. Layouts animate while they run, and the *Stop* button or **Esc** stops them where they are. They're all under the *Layout* menu too, and every layout is one step for undo.

| command | what it does |
|---|---|
| `forceAtlas2Layout()` | Gephi's force-directed layout. Options: `linLog=true` (tighter clusters), `preventOverlap=true`, `gravity=`, `scaling=`, `dissuadeHubs=true`, `edgeWeightInfluence=`, `iterations=` |
| `gemLayout()` | GEM, a fast force-directed layout |
| `frLayout()` | Fruchterman–Reingold |
| `kkLayout()`, `weightedKKLayout()` | Kamada–Kawai (spring distances follow graph distance) |
| `springLayout()`, `weightedSpringLayout(min, max)`, `isomLayout()`, `physicsLayout()` | other force and self-organising layouts |
| `mdsLayout()` | multidimensional scaling of the graph distances |
| `sugiyamaLayout()`, `dagLayout()` | layered drawing for hierarchies |
| `treeLayout(root)` | a tree (`horizontal=true` for left to right) |
| `radialLayout(v1)` | rings around a node by distance |
| `circleLayout()` | a circle; `circleLayout(dept)` makes one circle per group |
| `hiveLayout(axis, position)` | a hive plot (below) |
| `scatterLayout(xfield, yfield)` | a scatter plot of two fields |
| `randomLayout()` | random positions |
| `drlLayout()`, `lglLayout()`, `umapLayout()` | for large graphs (igraph, section 7.9): DrL (tens of thousands of nodes, clusters well apart), LGL (grown from a root), UMAP (tight islands) |
| `dhLayout()` | Davidson–Harel: even spacing, few crossings; small graphs |
| `bipartiteLayout(field)` | a two-mode network in two rows |
| `binPackLayout()` | packs the connected components side by side |
| `noverlapLayout()`, `labelAdjustLayout()` | nudge nodes until they (or their labels) don't overlap |
| `rotateLayout(90)`, `mirrorLayout()`, `rescaleLayout(0.5)`, `translateLayout(x, y)` | transform the current layout |

Nodes whose `fixed` field is true stay where they are. So you can pin a few nodes and lay out the rest around them:

```
v1.fixed = true
gemLayout()
```

Use `centerAfterLayout(false)` if you'd rather the view didn't re-centre after every layout.

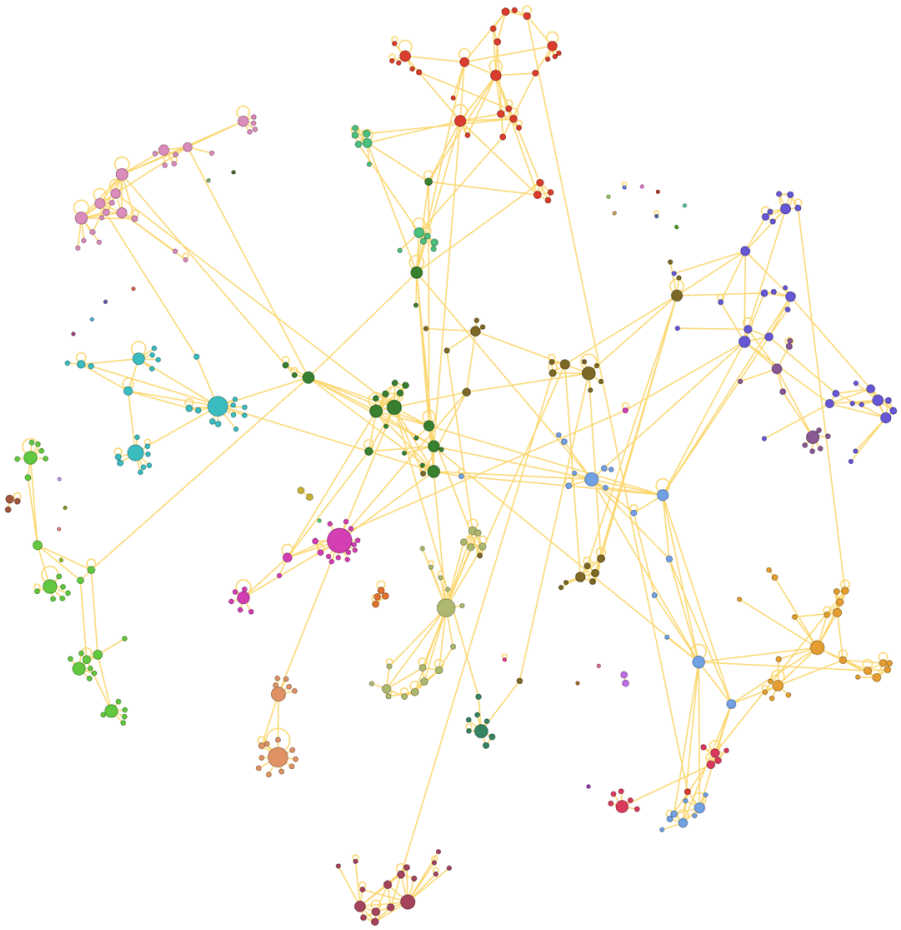


**Figure 8:** louvainClusters(); colorize(modularity_class); sizeBy(pagerank, 6, 40); forceAtlas2Layout(linLog=true, preventOverlap=true)

**Hive plots.** A hive plot puts each group on its own axis and places nodes along the axis by a value. Unlike force-directed layouts, the picture depends only on the data and not on chance, so two networks can be compared by their shapes.

```
hiveLayout(dept, totaldegree)            # an axis per department, ordered by degree
hiveLayout(dept, pagerank, axes=5)       # the 4 largest groups, plus "other"
```

The axes, their labels and the position note are drawn with the graph. They go away when you run another layout or load another graph.

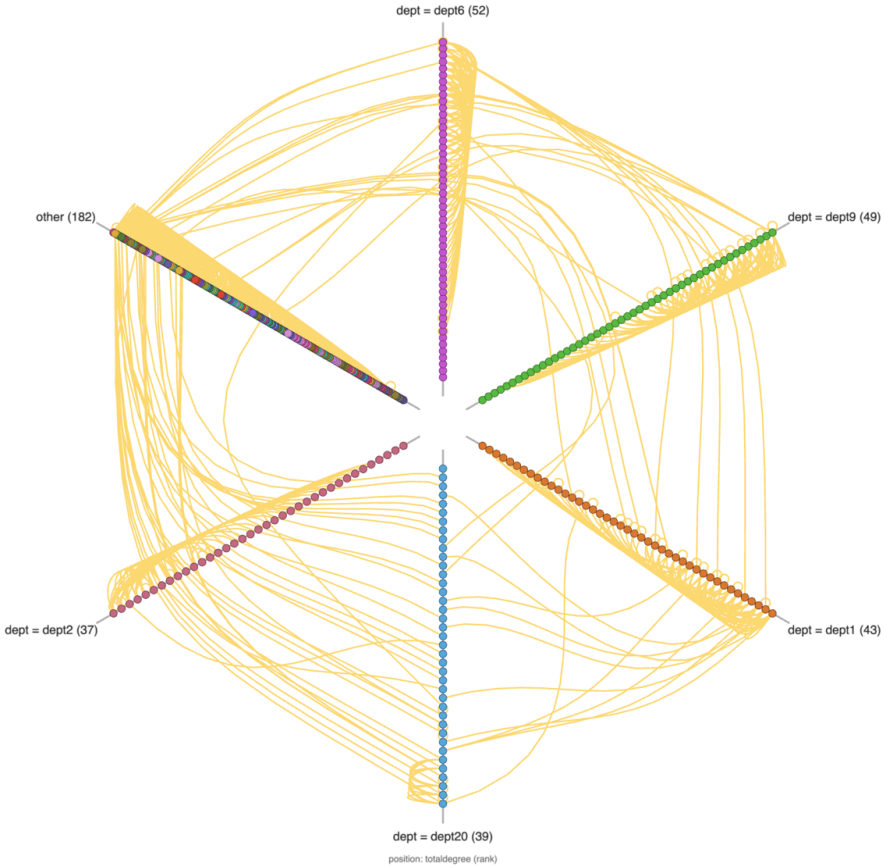



**Figure 9:** hiveLayout(dept, totaldegree)

**Scatter plots.** `scatterLayout(betweenness, pagerank)` places nodes by two fields, with axes. Dragging a node then edits those two fields (section 13).

**Old names.** The 2006 names still work. `random()` is the random layout, and `jfrLayout`, `jkkLayout1`, `jkkLayout2` and `jSpringLayout` are the JUNG versions of the force layouts. Every command is also a method of `g`, so `g.gemLayout()` works.

# 7. Analysis commands

## 7.1 Clustering and communities

Clustering commands return a set of sets (one per cluster) and, where it makes sense, add a field so you can colour by it:

```
louvainClusters()                   # communities; adds the field modularity_class
colorize(modularity_class)
modularity(modularity_class)        # how good the partition is
edgeBetweennessClusters(5)          # remove the 5 most "between" edges; the pieces
weakComponentClusters()             # connected components
biComponentClusters()               # biconnected components
groupsOfSize(dept, 40)              # groups with at least 40 members
```

## 7.2 Visualizing fields

The quickest way to see a field is to map it to colour, size, a label or a shape:

```
colorize(dept)                          # a distinct colour per value
colorBy(betweenness, yellow, red)       # a gradient over a number
colorBy(freq, yellow, red, 'log')       # scales: 'linear', 'log', 'sqrt', 'rank'
sizeBy(pagerank, 5, 40)                 # node size (or edge width) from a number
resizeLinear(totaldegree, 5, 30)        # the 2006 version
labelBy(dept)                           # labels from a field
shapeBy(dept)                           # a shape per value
borderBy(dept)                          # a border colour per value
legendFor(dept)                         # a legend for the current colours
GradientLegend(yellow, red, betweenness.min, betweenness.max)
```

`colorize` and `colorBy` work on node fields and on edge fields (they colour whichever the field belongs to). `colorize(freq, blue, red)` with two colours behaves like `colorBy`.

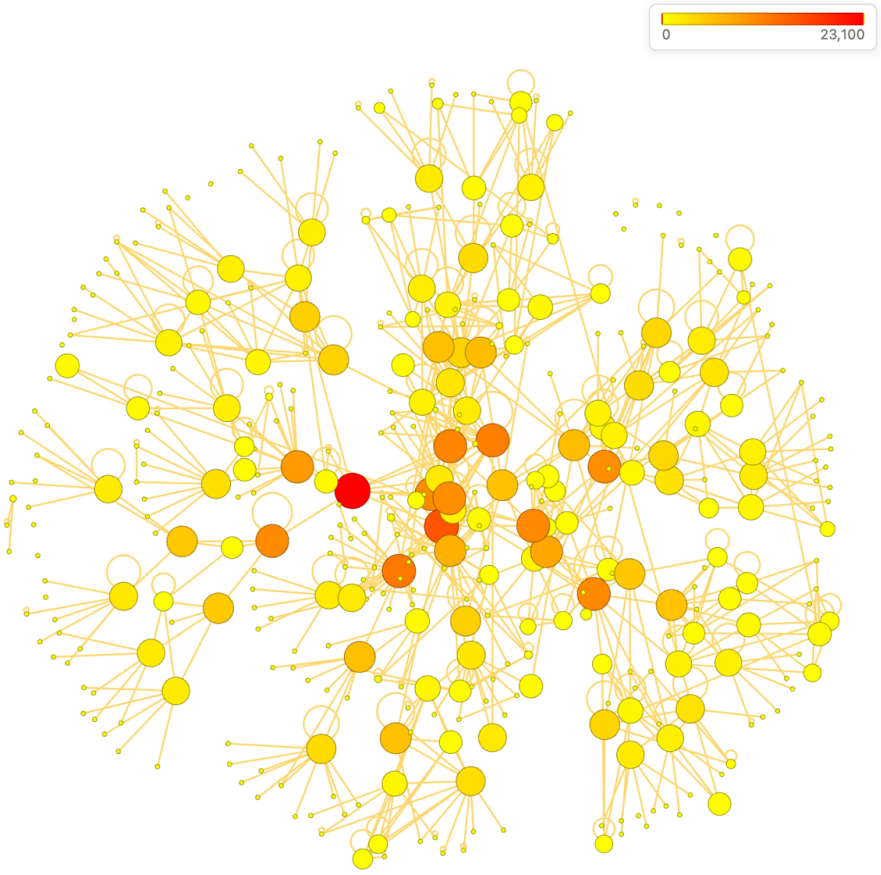



**Figure 10:** colorBy(betweenness, yellow, red); sizeBy(betweenness, 5, 40, 'log'); GradientLegend(…)

If you'd rather click than type, the **Style panel** (*View* → *Style panel*) does the same mappings with menus: pick nodes or edges, a property, a field and a mapping. It shows the command it runs. You can save a set of mappings as a named style, and `defineStyle`/`applyStyle` do the same from code:

```
defineStyle('by dept', ['colorize(dept)', 'legendFor(dept)'])
applyStyle('by dept')
```

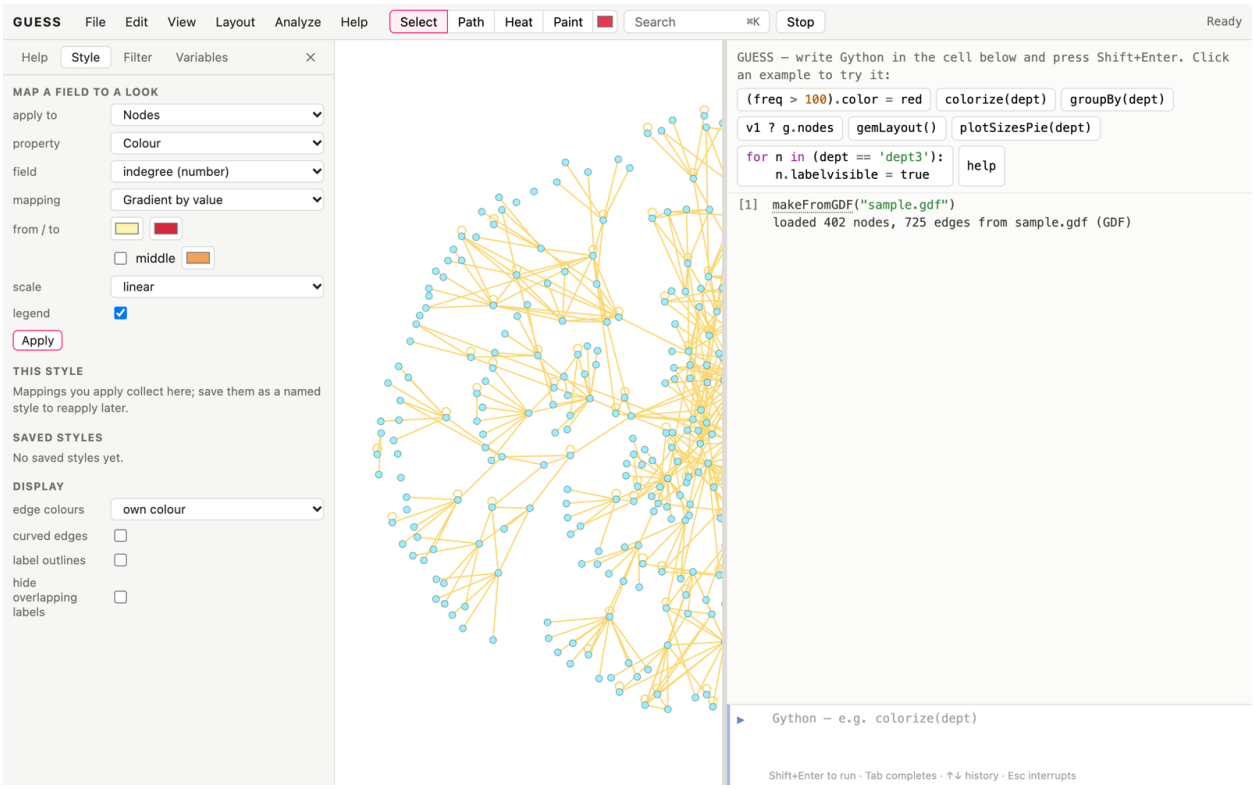



**Figure 11:** The Style panel.

A few more ways to show values on nodes:

```
pieChart([indegree, outdegree])     # each node becomes a pie of several fields
pieNeighbors(dept)                  # a pie of the neighbours' departments
pieOff()
bundleEdges(dept)                   # bundle edges that run between the same groups
unbundleEdges()
```

## 7.3 Field, graph, node and edge statistics

These node fields are computed the first time you use them. Use them like any other field:

| field | meaning |
|---|---|
| `indegree`, `outdegree`, `totaldegree` | degree |
| `degrank` | degree as a fraction of the total degree |
| `strength` | weighted degree (sum of edge weights) |
| `pagerank` | PageRank |
| `hub`, `authority` (`hits`) | HITS scores |
| `eigenvector` | eigenvector centrality |
| `betweenness` | betweenness centrality (also an edge field) |
| `rwbetweenness` | random-walk (current-flow) betweenness |
| `closeness`, `harmonic` | closeness and harmonic centrality |
| `eccentricity` | the greatest distance to another node |
| `clustering`, `triangles` | local clustering coefficient, triangle count |

| field | meaning |
|---|---|
| `coreness` | k-core number |
| `component`, `strongcomponent` | which (strongly) connected component |
| `constraint`, `effsize` | Burt's structural holes: low constraint and a large effective size mark brokers |
| `generation` | the step in a directed hierarchy (0 = nothing points to it) |

```
(pagerank > 0.01).labelvisible = true
sortBy(betweenness)[-10:]           # the ten most "between" nodes
recompute()                         # after editing the graph
```

Whole-graph measures are commands: `density()`, `diameter()`, `averagePathLength()`, `averageClustering()`, `transitivity()`, `assortativity()` (do hubs link to hubs?), `reciprocity()` (directed edges), `triadCensus()`, and `graphStats()` for a summary. `distance(v1, v2)` is the distance between two nodes on the screen.

### 7.4 Paths and filters

```
shortestPath(v1, v14).color = red       # nodes and edges of a shortest path
v1.unweightedShortestPath(v14)          # the path's edges (2006 form)
v1.dijkstraShortestPath(v14)            # weighted by the weight field
heatMap(v1)                             # colour everyone by distance from v1
showOnly(kCore(3))                      # the 3-core
showOnly(egoNetwork(v1, 2))             # everyone within 2 steps of v1
showOnly(giantComponent())
showOnly(intraEdges(dept))              # edges inside departments
interEdges(dept)                        # edges between departments
showAll()
```

The filters return sets, so they combine with queries: `showOnly(kCore(3) & (dept == 'dept1'))`. The **Filter panel** (*View* → *Filter panel*) stacks filters with sliders and checkboxes, if you'd rather build them by hand.

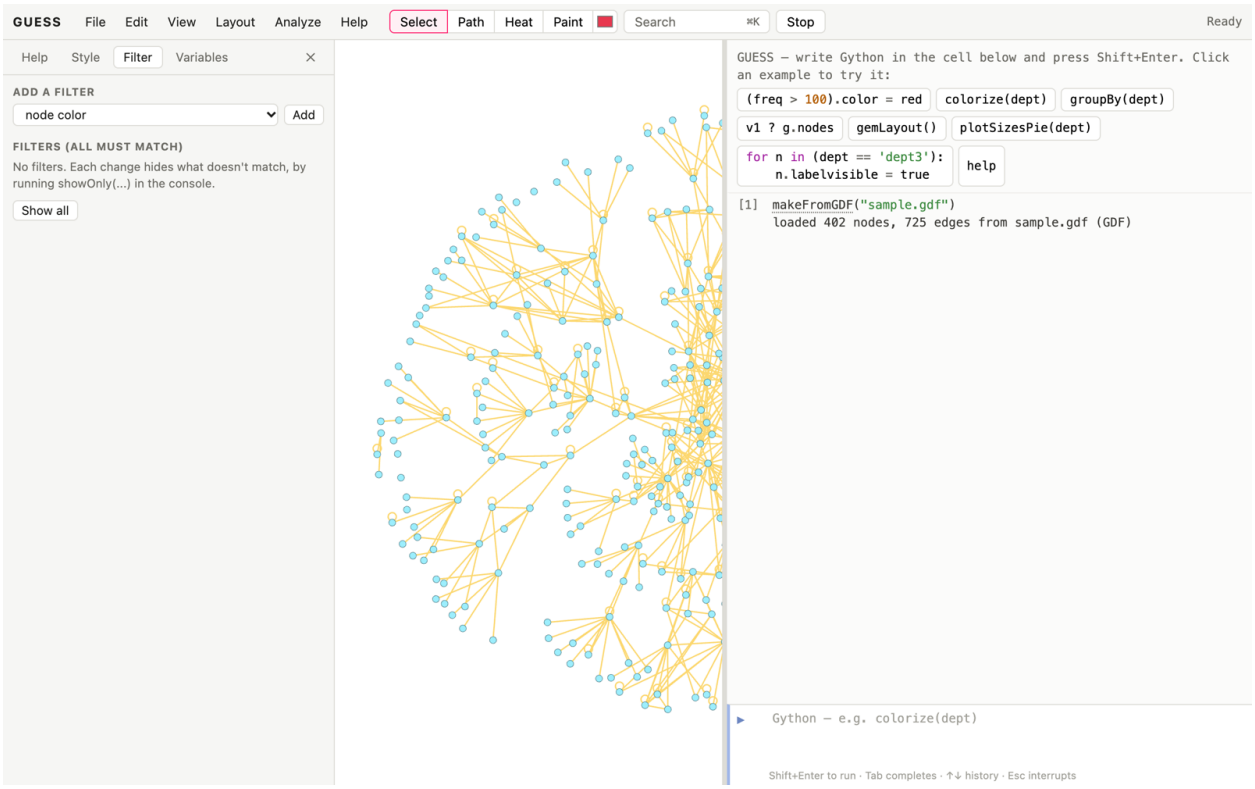



**Figure 12:** The Filter panel.

## 7.5 Charts

Charts are linked to the graph. Hover over a bar or a slice and its nodes light up; click and they're selected.

```
plotSizesPie(dept)          # how many in each department
plotSizes(dept)             # the same, as bars
plotHistogram(pagerank)     # distribution of a number
plotDistrib(totaldegree)    # each value in rank order
```

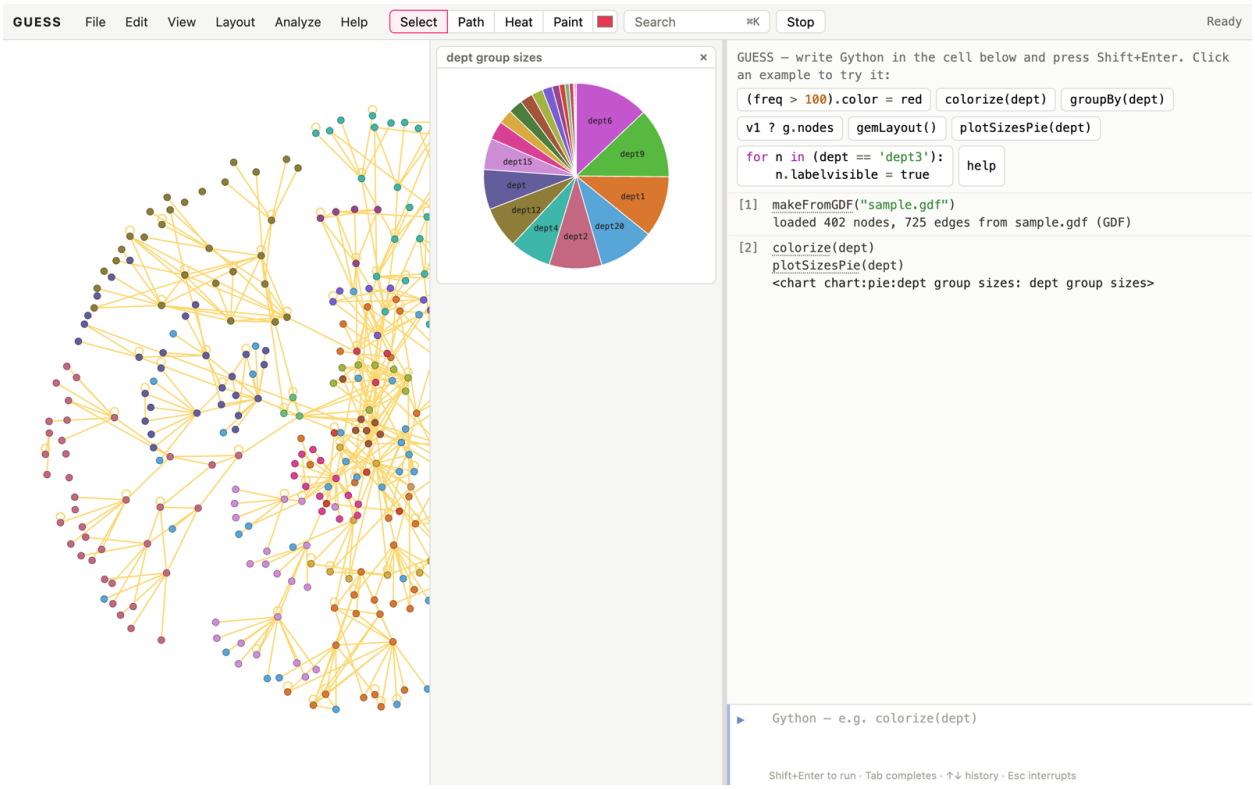



**Figure 13:** plotSizesPie(dept)

## 7.6 Random graph generation

See section 4.5 for the generators. A typical use is to compare a network with random networks of the same size:

```
makeErdosRenyiRandom(400, 0.009)
averageClustering()
```

## 7.7 Structure, mixing and comparing partitions

These commands return sets you can colour, hide or show, or a single number:

```
makeFromGDF("sample.gdf")                  # back to the sample graph
bridges().color = red                      # edges whose removal splits the graph
cutNodes().size = 30                       # nodes whose removal splits it
showOnly(spanningTree(maximum=True))       # the backbone of strongest ties
showAll()
sortBy(constraint)[:5]                     # brokers (structural holes)
assortativity(dept)                        # 1: ties stay in a department; 0: random
mixingMatrix(dept)                         # edges between each pair of departments
louvainClusters()
compareClusters(modularity_class, dept)    # do the communities match departments?
```

`compareClusters` gives normalized mutual information by default (0 to 1); `method="ari"` (adjusted Rand), `"rand"` and `"vi"` are the alternatives.

For directed graphs such as citations or dependencies, `ancestors(n)` and `descendants(n)` follow the arrows, the `generation` field gives each node's step in the hierarchy, `transitiveReduction()` returns the edges that longer paths already imply (hide them to see the backbone), and `feedbackEdges()` returns edges that close cycles.

## 7.8 networkx from the console

networkx is loaded with GUESS, so any of its algorithms is one line away. `toNetworkX()` gives the graph with its fields as attributes (node keys are node names). `setField` stores a result as a field: a node dictionary, an edge dictionary, or a list of communities.

```
import networkx as nx
G = toNetworkX()
setField(nx.average_neighbor_degree(G), "nbrdeg")
setField(nx.community.greedy_modularity_communities(G), "greedy")
colorize(greedy)
```

`fromNetworkX` goes the other way. It turns attributes into fields and uses `pos` (or `x` and `y`) for positions, with a spring layout when there are none. Given a name, it opens the graph in a new tab and switches to it; the tabs above the graph switch back.

```
fromNetworkX(nx.karate_club_graph(), "karate")
colorize(club)
use("main")                                  # back to the first graph
```

## 7.9 igraph: faster statistics and more methods

GUESS also loads igraph, a graph library written in C, a few seconds after it starts. Once it is there, betweenness, closeness, harmonic, eccentricity, diameter, clustering, triangles, coreness and components use it. The numbers are the same and large graphs are many times faster. `useIgraph(false)` switches back to the networkx and numpy versions, for comparison.

igraph also adds community methods. Each stores its result in a field (`leiden_class`, `infomap_class`, …; 0 is the largest community) and prints the modularity:

| command | method |
|---|---|
| `leidenClusters(resolution=1)` | Leiden: like Louvain, but every community is connected; `objective="CPM"` avoids modularity's resolution limit |
| `infomapClusters()` | Infomap: random-walk flow, best for directed graphs |
| `walktrapClusters(count)`, `fastGreedyClusters(count)` | hierarchies; `cutDendrogram(n)` re-cuts them into n groups |
| `labelPropagationClusters(field)` | very fast; given a field, spreads its values to the nodes without one |
| `eigenvectorClusters()`, `fluidClusters(k)`, `voronoiClusters()` | spectral; exactly k groups; around central nodes |
| `spinglassClusters()`, `optimalClusters()`, `cohesiveBlocks()` | small graphs: signed networks; the exact optimum; nested cohesive groups |

```
makeFromGDF("sample.gdf")
leidenClusters()
colorize(leiden_class)
compareClusters(leiden_class, dept)
similarTo(v1)                              # nodes whose neighbours overlap v1's
predictLinks(5)                            # likely missing edges
findPattern([("a", "b"), ("b", "c"), ("c", "a")])   # every triangle
minCut(v1, v14)                            # fewest edges separating two nodes
personalizedPagerank(v1)                   # relevance to v1, in the field ppr
largestCliques()
```

Also: `maxFlow(a, b)` (writes an edge field `flow`), `dominatorTree(root)`, `maxMatching()`, the node field `knn` (average degree of the neighbours), and two commands that don't need igraph: `simplify()` merges parallel edges, and `rewire()` makes a degree-preserving random copy in a new tab, to check whether a pattern is more than chance.

# 8. Modifying graphs

Let's take a few nodes out of the graph:

```
remove(totaldegree == 0)       # remove the isolated nodes
remove(v4)                     # a node takes its edges with it
remove(freq < 10)              # edges can be removed too
```

Removed elements go to `g.deleted`, so `add(g.deleted)` puts everything back. As in the 2006 version, a removed node keeps its name, so this works:

```
removeNode(v10)
addNode(v10)
```

`removeComplete(x)` removes something for good (it doesn't go to `g.deleted`). Undo works either way.

Other structural commands:

```
copyNode(v128, 'v128copy', color=red)    # a copy with the same fields
removeDisconnected()        # nodes with no edges
removeLeaves()              # nodes with one edge
removeSelfLoops()
shortcutNodes()             # replace nodes of degree 2 by a single edge
condenseGraph()             # remove leaves and shortcut nodes until nothing changes
```

**Fields.** Declare a new field before you use it. It then has a value on every node (or edge):

```
addNodeField("seniority", INTEGER, 0)
addEdgeField("kind", VARCHAR, "email")
v128.seniority = 10
```

The types are `INTEGER`, `DOUBLE`, `VARCHAR` and `BOOLEAN`, as in GDF (`Types.INTEGER` from 2006 also works).

**Grouping.** You can collapse a set of nodes into one *meta-node* and expand it again. This is useful for showing the structure between groups:

```
collapseBy(dept)           # one node per department, edges merged
expandAll()
collapse(dept == 'dept3', 'dept3group')
expand(dept3group)
```

A meta-node's size reflects how many nodes it holds, and double-clicking it expands it.

# 9. Output commands

| command | output |
|---|---|
| `exportPNG("g.png")`, `exportJPG(...)`, `exportGIF(...)` | an image of the view |
| `exportPDF("g.pdf")`, `exportPS(...)`, `exportEPS(...)` | an image of the view in a PDF / PostScript page |
| `exportSVG("g.svg")` | the whole graph as vector graphics |
| `exportGDF("g.gdf")`, `exportGEXF(...)`, `exportGraphML(...)`, `exportGML(...)`, `exportDOT(...)`, `exportCSV(...)` | the graph and its fields |
| `saveSession("s.guess")` | everything: graph, fields, states, styles, legends, hulls, view, slides |
| `saveHistory("work.py")` | every command you ran, as a script |
| `startMovie(30, "m.webm")` … `stopMovie()` | a movie of the display |

In a browser, "saving a file" means downloading it, so these files go to your downloads folder. The *File* menu has the common ones, plus:

- **Export notebook** writes the session as a Jupyter notebook (one cell per command).
- **Export report** writes an HTML page with the commands, their output, and a picture of the graph.
- **Share link** makes a URL that contains the whole session. Anyone who opens it sees your graph as you left it. **Share as viewer** does the same, but hides the console.

`history()` lists every command you've run this session. `log("work.py")` starts writing commands to a file as you go, and `stoplog()` stops.

## 10. Several graphs

Sometimes you want to look at part of a graph on its own, or compare two graphs. Each graph lives in a tab above the display:

```
subgraph(dept == 'dept3', 'dept3')        # a copy of these nodes and their edges, in a new t
use('main')                               # switch back to the first graph
renameGraph('office')                     # rename the current graph
newGraph('scratch')                       # an empty graph
graphNames()
use('office')
closeGraph('scratch')                     # (not the current one)
```

Clicking a tab switches to it as well. Commands always apply to the current graph, and `g` always refers to it.

# 11. States and animations

A **state** is a snapshot of the graph: which nodes and edges exist, and every field value. Save one with `ss`, change things, and load it again with `ls` or animate back to it with `morph`:

```
ss('before')
colorize(dept)
circleLayout()
morph('before', 2000)       # animate back over 2 seconds
ls('before')                # jump back
getStates()
```

During a morph, nodes and edges that only exist in one of the two states fade in or out. `setNodeDisappearBy(.2)`, `setNodeAppearAfter(.8)`, `setEdgeDisappearBy(…)` and `setEdgeAppearAfter(…)` say when, as a fraction of the morph. (The 2006 form `Morpher.setNodeDisappearBy(.2)` also works.)

States can be saved under numbers too (`ss(1999)`, `ss(2000)`), which suits a series of snapshots over time.

## 11.1 State-sensitive queries

A field subscripted with a state reads the values in that state:

```
ss('before')
(freq > 100).color = red
freq['before'] > 100              # the edges that had freq > 100 in 'before'
v1['before'].color                # v1's colour then
g.deleted                         # what's been removed
weight['_deleted'] > 5            # the 2006 form: deleted edges with weight > 5
```

Saved states can't be changed: `v1['before'].x = 20` is an error.

## 11.2 Ranges and time

A state is a whole snapshot. When nodes and edges simply come and go over time, **ranges** are lighter. Give each element the times it exists:

```
addRange(v1, "1–5")
addRange(v2, "3,7–9")
Node rcontains 4                # nodes whose range contains 4
Node roverlaps (2, 5)           # nodes whose range overlaps 2..5
Node rcontained (0, 10)         # nodes whose range lies inside 0..10
getRange(v1)
```

Ranges are closed intervals. "3-5" includes 3 and 5. If your nodes have a text field called `range` holding ranges like "1-5" (or another text field you name with `Node.setRangeField(when)`), the range operators use it for nodes without `addRange`.

Many graphs instead have a number for when each node appeared, such as a year. `showTime` shows the graph at a moment. Let's give the sample graph a (made-up) year for each person:

```
addNodeField('year', INTEGER, 0)
for n in g.nodes: n.year = 1990 + int(n.name[1:]) % 20
showTime(1995, year)            # nodes with year <= 1995 (and their edges)
showTime(1995, year, 5)         # only the last five years
showTime(3)                     # by ranges: only what exists at time 3
```

The **timeline** (*View* → *Timeline*) gives you a slider and a play button for the same thing. It finds number fields that look like time and plays through them.

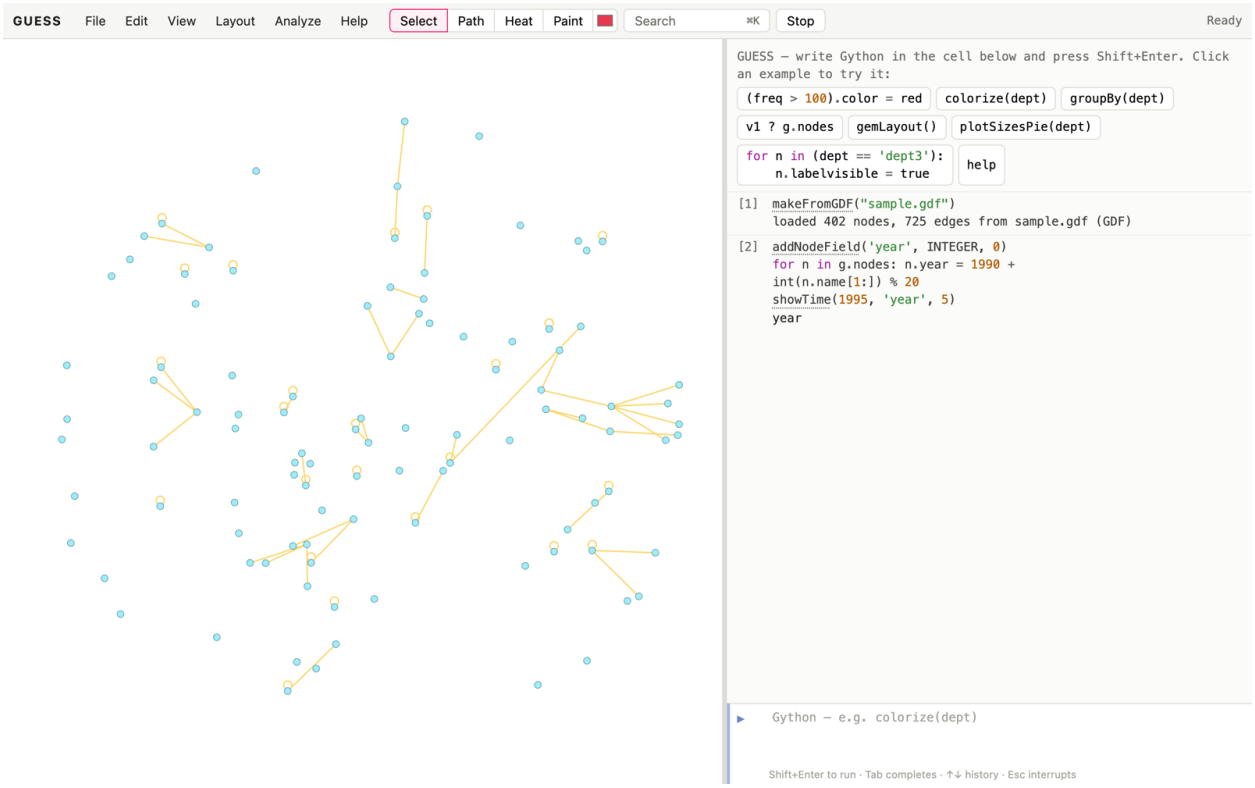



**Figure 14:** showTime(1995, 'year', 5): only nodes from 1991–1995 are shown.

# 12. Legends, hulls and the display

## 12.1 Legends

`legendFor(field)` makes a legend for the current colours of a field. `GradientLegend(start, end, min, max)` makes a colour ramp, and a fifth argument adds tick marks. For full control, build one yourself:

```
lg = Legend()
lg.add(v1, "a node")            # a node (as a prototype), a label
lg.add(sortBy(freq)[-1], "busiest")   # edges work too
lg.add(red, "important")        # or just a colour
```

Legends appear at the top right of the display, and they're included in exports and reports.

## 12.2 Convex hulls

A convex hull shades the area around a group of nodes:

```
h = createConvexHull(dept == 'dept4', "200,200,255,90")
for d in ('dept4', 'dept6', 'dept9'):
    grp = dept == d
    createConvexHull(grp, averageColor(grp[0].color, white))
removeConvexHull(h)
removeConvexHulls()
```

Hulls follow their nodes when you move them or run a layout.

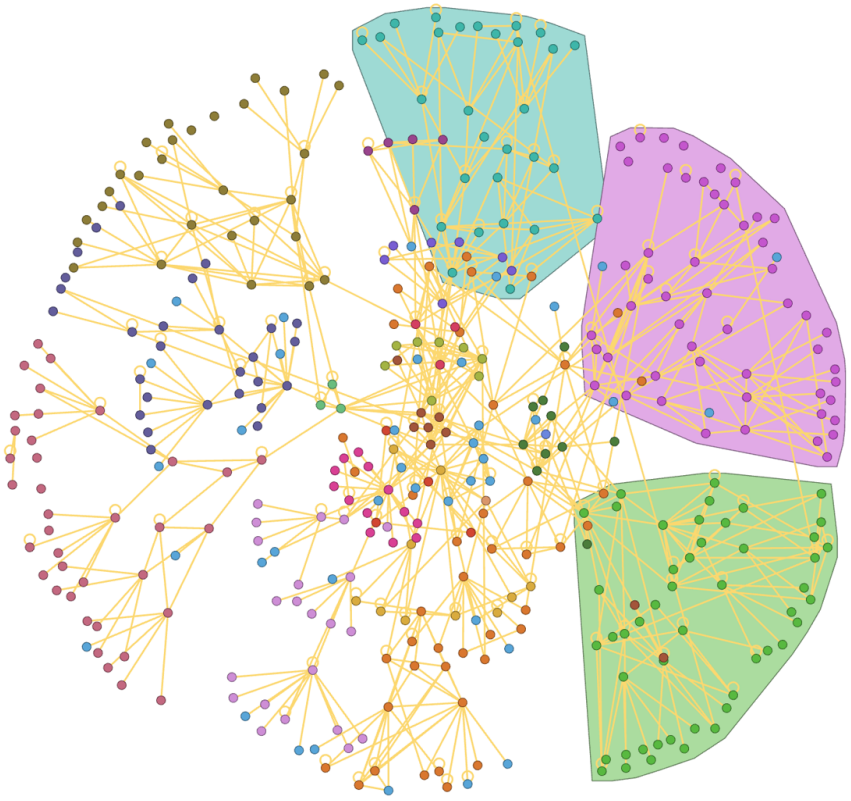


**Figure 15:** Hulls around three departments.

## 12.3 The display

- **Theme**: *View* → *Theme* is *Automatic* (follows your system), *Light* or *Dark*. The graph's background follows the theme, unless you set one with `setDisplayBackground(black)`. `setDisplayBackground(None)` follows the theme again.
- **Edges**: *View* → *Curved edges* or `curvedEdges()`. `setEdgeColorMode("mixed")` draws each edge in the colours of its two ends. Parallel edges between the same two nodes are always drawn as separate curves.
- **Labels**: `labelOutlines()` draws labels with a halo so they read on any background. `hideOverlappingLabels()` drops labels that would collide, keeping the bigger nodes' labels.
- **Background image**: `setBackgroundImage("map.png", x, y)` puts an image under the graph (a map, say), and `removeBackgroundImage()` takes it away.
- **Arrows**: `overrideArrowLength(12)`, `overrideArrowWidth(6)`.

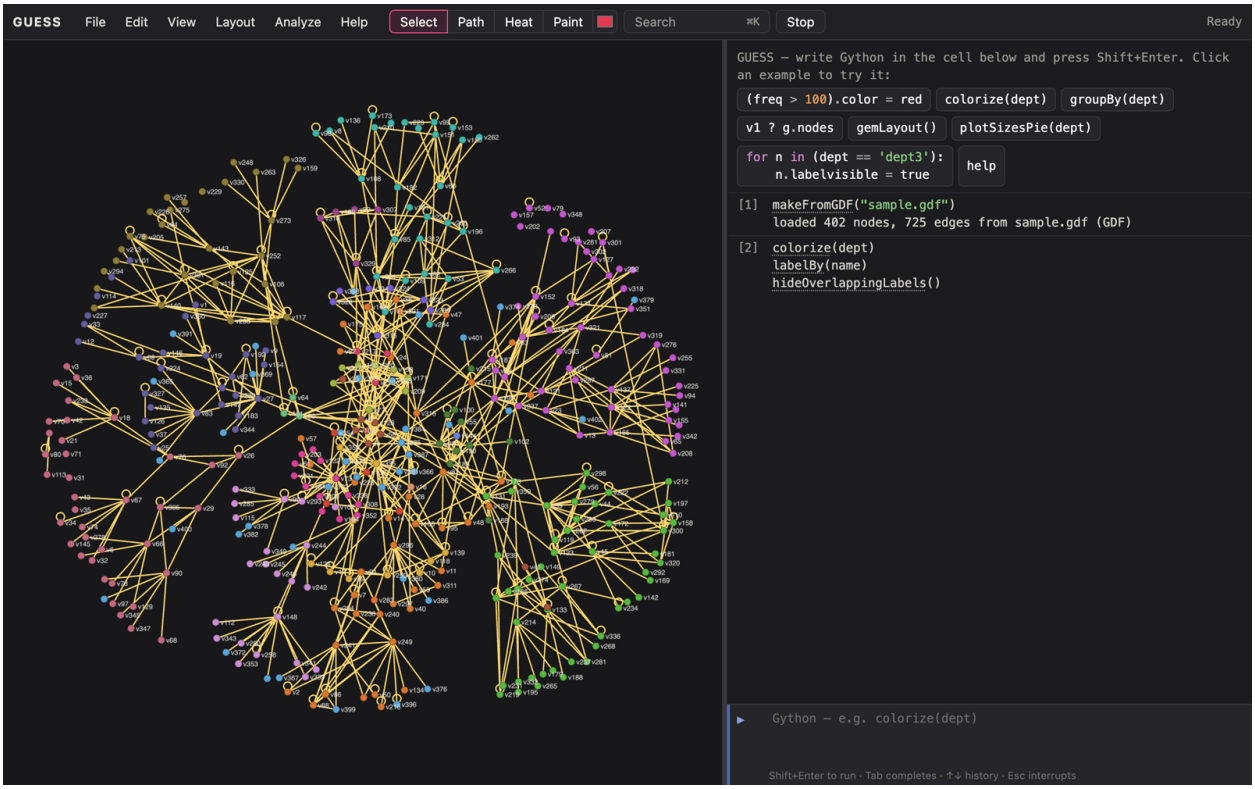



**Figure 16:** Dark theme, with colorize(dept), labelBy(name) and hideOverlappingLabels().

# 13. Direct manipulation

The mouse does more than pan and zoom. *Help → Mouse and gestures* shows all of this with pictures, and a tip appears the first time a gesture becomes useful.

## 13.1 Looking

| gesture | does |
|---|---|
| drag the background | pan |
| wheel or pinch | zoom |
| double-click the background | fit the graph |
| hover a node or edge | its fields in a tooltip; an edge rings its two endpoints |
| click a node | focus on it and its neighbours (click again or Esc to clear) |
| double-click a node or edge | zoom to it |
| hover a node or edge in the console output | highlight it in the graph; click to select and zoom to it |

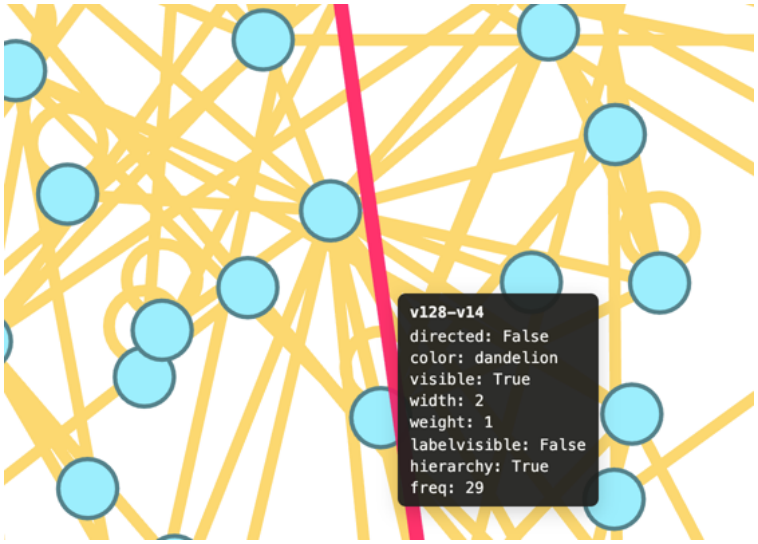



**Figure 17:** Hovering an edge rings both endpoints.

## 13.2 Selecting

| gesture | does |
|---|---|
| Shift-drag | box select |
| Alt/Option-drag | lasso select |
| Cmd/Ctrl with a box or lasso | add to the selection |
| Shift-click a node | add it to (or remove it from) the selection |
| Esc, or click the background | clear the selection |
| right-click | a menu for the node, the edge, or the whole selection |

| gesture | does |
|---|---|
| Delete or Backspace | remove the selected nodes (also *Edit* → *Delete selected nodes*, or *Delete* in the right-click menu); undo brings them back |

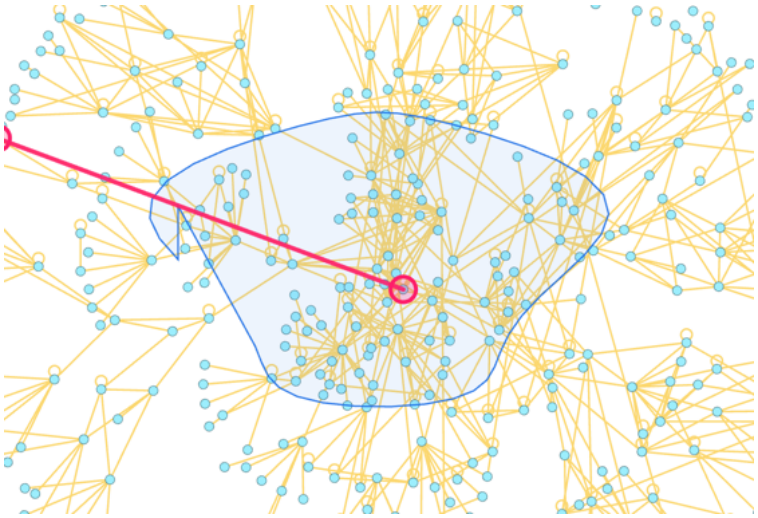


**Figure 18:** A lasso selection (Alt/Option-drag).

The selection is `sel` in the console (section 5.4). The right-click menu colours, sizes, labels, hides and removes, and *Assign to variable…* gives the selection a name you can use in code.

## 13.3 Dragging with meaning

Dragging a node moves it, and the selection moves with it if the node is selected. Each drag is recorded as an ordinary command, such as `v12.x = 310.5; v12.y = -42`. It is one undo step, and `history()`, `saveHistory()`, `log()` and notebook export include it, so a saved script puts the nodes back where you left them. To keep the console readable, these commands are hidden. *View* → *Show node moves in the console* shows them.

After some commands, a drag also *means* something. GUESS knows what the arrangement on the screen stands for, so it can turn your drag into the change you meant:

- **Reordering a circle.** After `circleLayout()`, nodes slide around the circle and the others make room. Dropping runs `ringMove(node, position)`.
- **Adding an edge.** Alt/Option-drag from one node to another adds an edge.
- **Editing values.** After `scatterLayout(x, y)`, dragging a node changes its two fields.

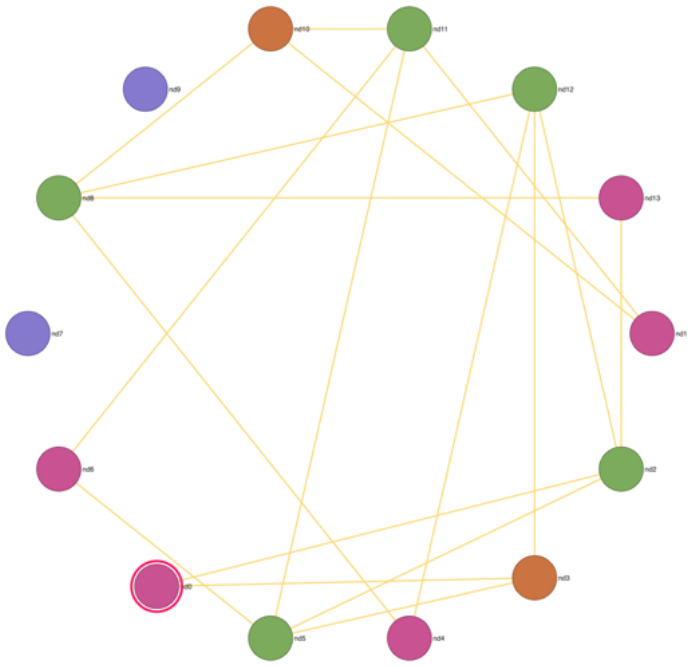


**Figure 19:** After circleLayout(), a dragged node slides around the ring and the others make room.

Each of these drops appears in the console as a visible command (for example `ringMove(v128, 4)`). So you can see what happened, undo it, or copy it into a script.

## 13.4 The click tools

The tools next to the menus turn clicks into commands:

- **Select** (the default) is everything above.
- **Path**: click two nodes to colour a shortest path between them.
- **Heat**: click a node for a heat map of the distance from it.
- **Paint**: click nodes to paint them with the colour next to the tools. Shift-click paints the neighbours too.

Each click runs a command in the console (`shortestPath(v1, v14)`, `heatMap(v3)`, `paint(v5, red)`), so you can edit it and run it again.

## 13.5 Responding to clicks in code

You can also attach your own code to the mouse:

```
graphevents.clickNode = lambda n: center([n] + n.getNeighbors())
graphevents.mouseEnterNode = lambda n: setattr(n, 'labelvisible', true)
graphevents.mouseLeaveNode = lambda n: setattr(n, 'labelvisible', false)
```

The events are `clickNode`, `clickEdge`, `shiftClickNode`, `shiftClickEdge`, `mouseEnterNode`, `mouseLeaveNode`, `mouseEnterEdge` and `mouseLeaveEdge`. Setting one to `None` turns it off.

`vf.defaultNodeHighlights(false)`, `vf.defaultNodeZooming(false)`, `vf.defaultEdgeHighlights(false)` and `vf.defaultEdgeZooming(false)` turn off the built-in hover highlight and double-click zoom. Do this when your own handlers should be the only thing that happens. `v.getLastClickedPosition()` gives the last place clicked, in graph coordinates.

You can add items to the right-click menu, too:

```
def paintRed(n):
    n.color = red

item = NodeEditorPopup.addItem("Paint red")
item.menuEvent = paintRed
```

`EdgeEditorPopup` adds to the edge menu and `GraphElementPopup` to both. When several things are selected, the function receives the selection.

# 14. The console, history and undo

| key / action | does |
|---|---|
| Shift+Enter, Cmd/Ctrl+Enter | run the cell |
| Enter | a new line |
| Tab | complete commands, fields, nodes and attributes |
| ↑ / ↓ on the first / last line | earlier / later commands |
| Esc, or *Stop* | interrupt running code |
| Ctrl+L | clear the output |
| Shift+Cmd/Ctrl+F | find in the console output |
| **Edit** on a cell (or double-click its code) | copy it into the working cell |
| **Run again** on a cell | run it again, as a new cell at the bottom |
| **Watch** on a cell | re-run it whenever the graph changes |
| `[n]` | fold or unfold a cell's output |

*View → Vim keys in the code cell* switches the editor to vim bindings.

**Undo.** Every command is one step: a layout, a file load, `colorize`, a node drag, a table edit. *Edit → Undo*, Cmd/Ctrl+Z (when the code cell isn't focused) or `undo()` reverses the last step, up to 50 steps back. Redo is Shift+Cmd/Ctrl+Z, Ctrl+Y or `redo()`. Undone cells are struck through until you redo them. Undo covers the graph, the fields, states, ranges, hulls, legends, styles and the display. It doesn't cover Python variables you made, or files you've downloaded.

**History.** The cells on the screen are the session's history, in the order you ran them. `history()` lists them and `saveHistory("work.py")` downloads them as a script that rebuilds your work. Errors have a red bar, and the line that failed is underlined.

# 15. Modifying the interface / expanding GUESS

GUESS can be extended in Gython itself: new commands, panels of controls, and handlers for the mouse. In 2006 this meant Java Swing. Now it's a few small Python classes.

## 15.1 Example 1: a simple button

```
def bigger():
    g.nodes.size = 20

b = Button("Bigger nodes", onClick=bigger)
ui.dock(Panel("Tools", b))
```

`Panel(title, *widgets)` makes a panel, and `ui.dock(panel)` docks it below the graph (`position="right"` docks it to the right). The widgets are `Button`, `Label`, `Slider`, `Checkbox`, `Dropdown` and `TextField`.

## 15.2 Example 2: a threshold slider

This slider hides edges whose `freq` is below the threshold, and a label counts what's left:

```
lab = Label('')
def hideshow(val):
    g.edges.visible = true
    (freq < val).visible = false
    lab.text = '%d edges visible' % len(Edge.visible == true)

s = Slider(freq.min, freq.max, value=100, label='Minimum frequency', onChange=hideshow)
ui.dock(Panel('Threshold', s, lab))
hideshow(100)
```

`onChange` runs when you let go of the slider. `onSlide` runs continuously while you drag. (`Edge.visible == true` is a query on the edge field `visible`. `Node` and `Edge` pick which kind of field when the name is shared.)

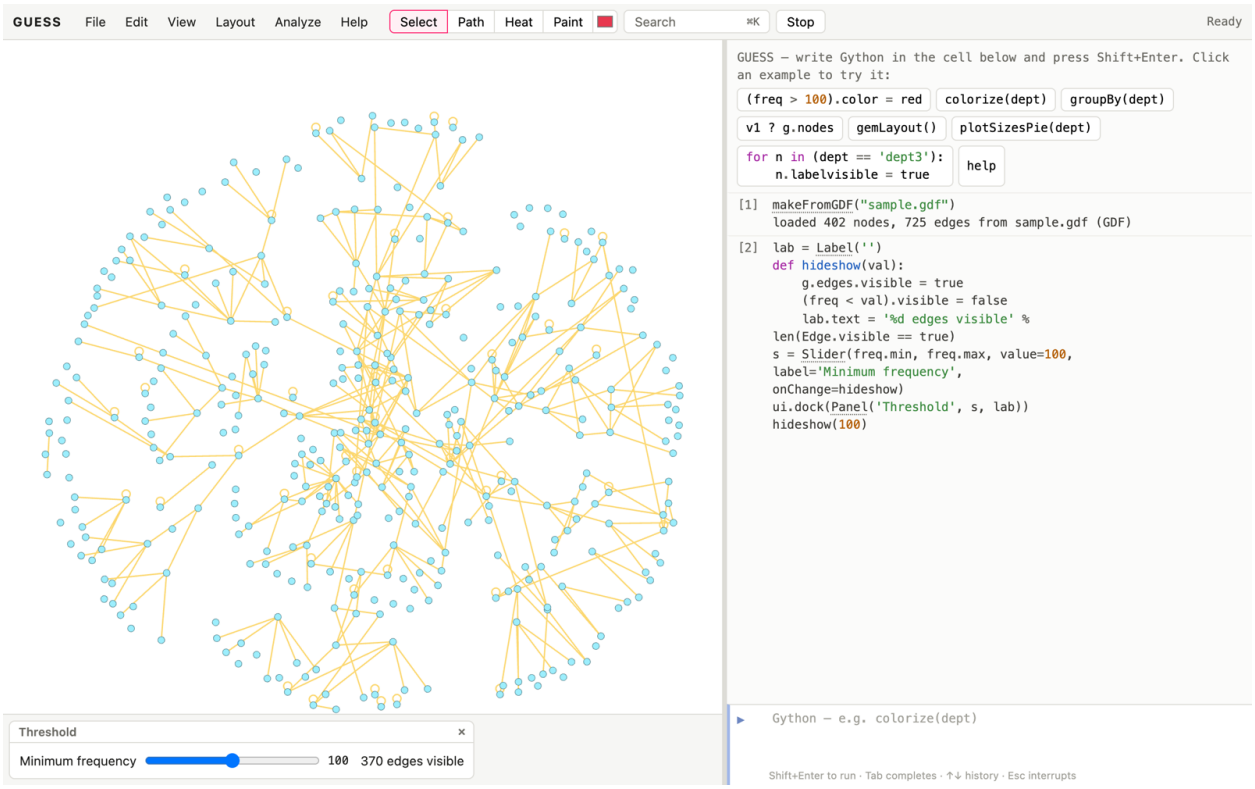



**Figure 20:** The threshold slider docked below the graph.

## 15.3 Example 3: a network monitor

A timer runs code every few seconds, for instance to poll for new data:

```
def tick():
    v = randomColor()
    (dept == 'dept3').color = v

t = every(2, tick)        # every 2 seconds
t.stop()                  # or stopTimers() for all of them
after(5, gemLayout)       # once, in 5 seconds
```

A timer's code runs between your commands, so it never interrupts one. A handler that takes longer than its interval doesn't pile up behind itself.

## 15.4 Example 4: an app

`exportApp` packages the current session with a script that builds its interface:

```
exportApp("threshold.guess", "ui.py")
```

Opening `index.html?load=threshold.guess&app=1` (with the file on the same server) shows the graph and its panels without the console. *File* → *Share as viewer* does the same thing as a single link. This replaces the applets of 2006.

## 15.5 Code bits

- `execfile("file.py")` runs a script you've opened.
- `openURL(url)` opens a web page, and `getAppletContext().showDocument(url)` (2006) does the same.
- `installPackage("name")` installs a pure-Python package from PyPI into the page, so you can `import` it.
- Presentations: `addSlide("note")` saves the current view as a slide, and *View* → *Present* steps through them with the arrow keys. `showSlide(2)`, `moveSlide(…)`, `noteSlide(…)` and `removeSlide(…)` edit them.

# 16. Sharing and deploying

## 16.1 Sessions

`saveSession("work.guess")` (*File* → *Save*) saves everything about the graph: the nodes, edges and fields, the states, styles, legends, hulls, slides, groups, the other graph tabs and the view. Opening the file restores it. Python variables aren't saved. If you need them, save your history as a script.

## 16.2 Links

*File* → *Share link* puts the whole session, compressed, into a URL. Nothing is uploaded: the graph travels in the link itself, so links to big graphs are long. *Share as viewer* adds `&app=1`, which hides the console.

## 16.3 Notebooks and reports

*File* → *Export notebook* writes a Jupyter notebook of your commands. *File* → *Export report* writes a self-contained HTML page with the commands, their output and a picture of the graph. Use it to send someone what you found.

## 16.4 Deploying GUESS

GUESS is a set of static files, so any web server can host it. From a copy of the source:

```
cd web
npm install
npm run bundle              # web/bundle/guess-web-<version>/ and .zip, .tar.gz
npm run bundle -- --offline   # the same, with the Python runtime included
```

Copy the contents of the `guess-web-<version>` folder (or unzip the archive) to the server. The site includes this manual under `manuals/`, which *Help* → *Manual* opens. A normal bundle loads the Python runtime from a CDN the first time it runs. An `--offline` bundle carries it (about 25 MB more), so it works with no internet access. The server must send two headers (`Cross-Origin-Opener-Policy` and `Cross-Origin-Embedder-Policy`) for the *Stop* button to interrupt running code. The bundle includes a fallback for servers that can't. `docs/DEPLOY.md` has ready-made settings for Apache, nginx, Caddy, Netlify and GitHub Pages.

To open a graph straight away, link to `index.html?load=mygraph.gdf`, with the file next to `index.html`. Add `&app=1` to hide the console.

# Appendix A. Colors

Colours can be given by name (`red`, `cornflowerblue`), as `"r,g,b"` or `"r,g,b,a"` strings (0–255), or as `"#rrggbb"`. The names are variables, so `v1.color = wildstrawberry` needs no quotes. `averageColor(c1, c2)`, `generateColors(start, end, n)`, `distinctColors(n)` and `randomColor()` make new colours.

The named colours from 2006 are below (case doesn't matter). The standard web colour names work as well.

| name | r, g, b | name | r, g, b | name | r, g, b |
|---|---|---|---|---|---|
| apricot | 251, 213, 184 | lavender | 251, 190, 254 | processblue | 46, 253, 253 |
| aquamarine | 115, 253, 217 | lfadedgreen | 241, 253, 230 | purple | 191, 106, 253 |
| bittersweet | 223, 45, 2 | lightcyan | 227, 253, 254 | rawsienna | 194, 1, 1 |
| black | 0, 0, 0 | lightgray | 192, 192, 192 | red | 255, 0, 0 |
| blue | 0, 0, 255 | lightgreen | 228, 253, 216 | redorange | 252, 130, 101 |
| bluegreen | 106, 253, 212 | lightmagenta | 251, 229, 254 | redviolet | 197, 13, 211 |
| blueviolet | 79, 66, 249 | lightorange | 252, 229, 216 | rhodamine | 250, 118, 253 |
| brickred | 218, 1, 2 | lightpurple | 227, 229, 253 | royalblue | 0, 187, 253 |
| brown | 168, 0, 1 | lightyellow | 252, 252, 202 | royalpurple | 130, 91, 253 |
| burntorange | 253, 184, 7 | limegreen | 188, 252, 9 | rubinered | 250, 14, 239 |
| cadetblue | 161, 174, 226 | lskyblue | 234, 247, 254 | salmon | 252, 181, 205 |
| canary | 251, 252, 187 | magenta | 255, 0, 255 | seagreen | 150, 253, 187 |
| carnationpink | 250, 163, 253 | mahogany | 209, 0, 2 | sepia | 147, 0, 1 |
| cerulean | 61, 240, 253 | maroon | 213, 0, 31 | skyblue | 162, 253, 240 |
| cornflowerblue | 156, 238, 253 | melon | 252, 192, 187 | springgreen | 221, 252, 134 |
| cyan | 0, 255, 255 | midnightblue | 0, 176, 198 | tan | 236, 198, 176 |
| dandelion | 252, 216, 112 | mulberry | 204, 82, 251 | tealblue | 95, 250, 208 |
| darkgray | 64, 64, 64 | navyblue | 57, 180, 254 | thistle | 237, 170, 253 |
| darkorchid | 199, 124, 230 | olivegreen | 13, 201, 7 | turquoise | 104, 253, 230 |
| emerald | 0, 253, 187 | orange | 255, 200, 0 | violet | 119, 99, 253 |
| forestgreen | 39, 239, 34 | orangered | 251, 11, 187 | violetred | 250, 121, 253 |
| fuchsia | 174, 47, 244 | orchid | 210, 161, 253 | white | 255, 255, 255 |
| goldenrod | 253, 241, 112 | peach | 252, 186, 148 | wildstrawberry | 251, 59, 203 |
| gray | 128, 128, 128 | periwinkle | 170, 178, 254 | yellow | 255, 255, 0 |
| green | 0, 255, 0 | pinegreen | 8, 222, 111 | yellowgreen | 196, 252, 139 |
| greenyellow | 236, 252, 151 | pink | 255, 175, 175 | yelloworange | 253, 198, 7 |
| junglegreen | 23, 253, 184 | plum | 182, 18, 253 | | |

# Appendix B. Gython and Python 3

Gython is Python 3 with these additions:

| addition | meaning |
|---|---|
| `field == value` (and `!=`, `<`, `>`, `<=`, `>=`, `like`) | a query: the set of matching nodes or edges |
| `set.field = value` | sets the field on every member |
| `a & b`, `a \| b` | intersection and union of sets |
| `->`, `<-`, `<->`, `-`, `?` | edges between nodes or sets |
| `Node rcontains x`, `roverlaps`, `rcontained` | range queries |
| `field['state']` | values in a saved state |

| addition | meaning |
|---|---|
| `true`, `false` | the same as `True`, `False` |
| a command name on its own (`center`) | calls it |
| node names, field names, commands, colours | predefined, protected variables |

Things to know if you're coming from the 2006 version (Jython, i.e. Python 2):

- `print` is a function: `print(v1)`, not `print v1`.
- `/` always divides exactly (`7/2` is 3.5). `//` is integer division.
- `except (A, B) as e`, not `except (A, B), e`.
- Strings are Unicode. There's no `unicode()`, and `str()` is all you need.
- Java isn't there: `import javax.swing` and friends don't exist (see section 15 for the widgets that replace them). Neither does the R interface.

## Appendix C. Changes from the 2006 version

Nearly everything in the 2006 manual still works, and its examples run unchanged except where Python 3 differs. The main differences:

- **Platform.** A web page instead of a Java application. No installation, no database files, no start-up dialogs. Work is autosaved in the browser, and `.guess` files hold sessions.
- **The console** is made of cells, with history, completion, undo and help.
- **Mouse.** Clicking a node focuses its neighbourhood instead of centring on it. Shift-drag selects instead of zooming. Zoom is the wheel or a pinch, not a right-drag. The five tool buttons (node, edge, hull and annotation editing) are replaced by the click tools and the drag gestures of section 13.
- **Output.** SWF, Java, CGM and EMF export are gone. Use SVG or PDF. Movies are WebM instead of QuickTime.
- **Interface building** uses `Panel`, `Button`, `Slider` and timers instead of Java Swing classes, threads and sockets.
- **Not available**: the R interface, applet signing and `jythonc`, the Prefuse and TouchGraph front ends, and command-line options. These are replaced by links and apps (section 16).
- **Multiple edges.** Any number of edges is allowed between two nodes, not just three.
- **Defaults.** Nodes are cornflowerblue ellipses and edges are dandelion, as the 2006 code had them (the 2006 manual says blue and green).

`docs/MANUAL_COMPARISON.md` goes through the 2006 manual section by section.

# Appendix D. Command reference

Every command, generated from the code's own documentation (the same text as `help(name)` and the Help panel).

## Graph structure, loading and saving, mapping, view

**`add(seq)`**

Re-add removed nodes/edges, e.g. add(g.deleted).

**`addDirectedEdge(node1, node2, **attrs)`**

Add a directed edge from node1 to node2.

**`addEdge(node1, node2, **attrs)`**

Add an undirected edge.

**`addEdgeField(fieldName, sqlType='VARCHAR', defaultValue=None)`**

Create an edge field, e.g. addEdgeField("kind", VARCHAR, "").

**`addNode(name, **attrs)`**

Add a node (or return the existing one with that name). addNode(vx) with a removed node puts it back, with its field values.

**`addNodeField(fieldName, sqlType='VARCHAR', defaultValue=None)`**

Create a node field, e.g. addNodeField("score", DOUBLE, 0).

**`addUndirectedEdge(node1, node2, **attrs)`**

Add an undirected edge between two nodes (by node or name).

**`averageEdgeColor(edges=None)`**

Color edges the average of their endpoint colors (all edges by default).

**`biComponentCluster()`**

Same as biComponentClusters() (the manual's spelling).

**`center(*d)`**

center() fits the whole graph; center(seq) zooms to the given elements.

**colorBy(field, start, end, scale='linear', middle=None)**

colorBy(field, startColor, endColor) colours elements along a gradient in proportion to a numeric field's value; scale may be "linear", "log", "sqrt" or "rank"; middle=color adds a midpoint colour. Compare colorize, which spaces distinct values evenly.

**colorize(field, *cs)**

colorize(field) gives each value its own clearly different colour (the largest group first; see distinctColors); colorize(field, start, end) or colorize(field, start, middle, end) a gradient over the sorted values.

**complement(seq)**

Everything (nodes if seq has nodes, edges if it has edges) not in seq.

**condenseGraph()**

removeDisconnected, removeSelfLoops, then alternate removeLeaves and shortcutNodes until nothing changes. Returns [removed, added].

**copyNode(template, name, **extra)**

Create node `name` with all field values copied from `template`.

**density()**

Edges divided by the number of possible undirected node pairs.

**distance(node1, node2)**

Euclidean distance between two nodes' positions.

**einverse(seq)**

All edges not in seq.

**execfile(filename)**

Run a Gython script file in the current namespace.

**exportCSV(filename='edges.csv', table='edges')**

exportCSV("edges.csv") writes the edge table (Source, Target, Type, Weight, Label and edge fields); exportCSV("nodes.csv", "nodes") the node table.

**exportDOT(filename='graph.dot', clusters='cluster', layout=True)**

exportDOT("graph.dot") writes the graph for Graphviz: positions (fixed nodes pinned with !), sizes, shapes, colours, labels, hidden elements and every user field; nodes are grouped into cluster subgraphs by the field named by clusters= (default `cluster`, e.g. clusters="modularity_class"). layout=False leaves positions out so Graphviz lays the graph out itself. makeFromDOT reads it back. Offered as a download.

**exportGDF(filename)**

Write the graph to a GDF file (offered as a download in the browser).

**exportGEXF(filename='graph.gexf')**

Write the graph as GEXF 1.3 for Gephi (offered as a download).

**exportGML(filename)**

Write the graph as GML (networkx, igraph, yEd, Cytoscape and Gephi read it): fields, positions, colours and sizes. Offered as a download.

**exportGraphML(filename)**

Write the graph as GraphML (offered as a download in the browser).

**findEdges(seq)**

The edges among the members of a (possibly mixed) set.

**findNodes(seq)**

The nodes among the members of a (possibly mixed) set.

**floatRange(a, b, inc)**

Like range() but for floats.

**getAppletContext()**

For old applet scripts: .showDocument(url) opens a browser window.

**getEdgeSchema()**

The edge fields.

**getNodeSchema()**

The node fields.

**groupAndSortBy(*d)**

Like groupBy, with the groups ordered by value.

**groupBy(*d)**

groupBy(field) or groupBy(seq, field): groups of elements sharing a value.

**help(obj=None)**

help() opens the help browser (searchable commands and examples; also F1 or Cmd/Ctrl+K for the command palette); help(command) prints that command's documentation and shows it in the browser.

**hideDisconnected()**

Hide nodes with no edges other than self loops.

**infowindow()**

Open the data table (the Jython version's information window).

**makeFromCSV(filename, nodes=None, directed=False)**

makeFromCSV("edges.csv") replaces the graph with a spreadsheet table as Gephi reads it: an edge table (Source, Target[, Type, Weight, Label, …]), an adjacency matrix or an adjacency list. makeFromCSV("edges.csv", "nodes.csv") also reads a node table (Id[, Label, …]). Opening just a node table adds its columns to the current graph's nodes (matched by Id). Other columns become fields; directed=true makes edges without a Type directed.

**makeFromDL(filename)**

Replace the current graph with a UCINET DL file (fullmatrix, edgelist1, nodelist1).

**makeFromDOT(filename)**

Replace the current graph with a Graphviz DOT file (.dot, .gv): node and edge statements, subgraphs (cluster_* members get a `cluster` field), default attributes; pos, color/fillcolor, label, width/height, shape, penwidth and weight map to GUESS, other attributes become fields.

**makeFromGDF(filename)**

Replace the current graph with the contents of a GDF file.

**makeFromGEXF(filename)**

Replace the current graph with a GEXF file (Gephi's format): attributes, colours, positions, sizes, edge weights and thickness.

**makeFromGML(filename)**

Replace the current graph with a GML file (Graph Modelling Language, as written by networkx, igraph, yEd and Cytoscape: labels, graphics x/y, fill colour, size and shape, and every other attribute as a field). For compatibility with GUESS 2006, a GraphML (XML) file works too.

**makeFromGraphML(filename)**

Replace the current graph with a GraphML file.

**makeFromPajek(filename)**

Replace the current graph with a Pajek .net file (nodes are named pajek).

**ninverse(seq)**

All nodes not in seq.

**readjustEdges()**

Kept for old scripts: parallel edges are always drawn as separate curves.

**remove(seq)**

Remove nodes and/or edges (a node takes its edges). Undo with add(...). As in the original, a removed node's name still refers to it, so add(name) or addNode(name) puts it back.

**removeComplete(seq)**

Remove nodes and/or edges permanently (they cannot be re-added).

**removeDisconnected()**

Remove (and return) nodes with no edges other than self loops.

**removeEdge(e)**

Remove one edge.

**removeLeaves()**

Repeatedly remove nodes of total degree <= 1. Returns what was removed.

**removeNode(n)**

Remove one node and its edges (undo with add(g.deleted) or undo()). Its name still refers to it, so addNode(n) puts it back.

**removeSelfLoops()**

Remove (and return) all edges whose endpoints are the same node.

**resizeLinear(field, start, end)**

Size nodes (or edge widths) by rank of field value, from start to end.

**resizeRandom(field, start, end)**

Give each distinct value of field a random size (or edge width) between start and end.

**setSynchronous(state=True)**

Kept for old scripts: layouts always run in order (animated, interruptible).

**shortcutNodes()**

Repeatedly replace degree-2 nodes by an edge between their neighbours. Returns [removed, added].

**sizeBy(field, smallest, largest, scale='linear')**

sizeBy(field, min, max) sizes nodes (or edge widths, for an edge field) in proportion to a numeric field's value, from min to max. scale may be "linear", "log", "sqrt" or "rank" (as in Gephi's ranking). Compare resizeLinear, which spaces distinct values evenly.

**skitter(field)**

Nodes around a circle ordered by field; radius shrinks with outdegree.

**sortBy(*d)**

sortBy(field) or sortBy(seq, field).

## Undo, redo, history and logging

**history()**

history() lists every command run this session, in order.

**log(filename='session.py')**

Start recording every successful command (save with stoplog()).

**redo()**

redo() re-applies the last command undone with undo().

**saveHistory(filename='history.py')**

saveHistory("work.py") downloads every command run this session as a script (run it again with execfile).

**stoplog()**

Stop recording and save the log (offered as a download); replay with execfile.

**undo()**

undo() reverses the most recent command that changed the graph, its fields, states, hulls, legends or display settings (Ctrl/Cmd+Z outside the code cell). Repeat to go further back (up to 50 steps); redo() re-applies. Variables and downloaded files are not undone.

## Statistics, communities, filters and paths

**averageClustering()**

The mean local clustering coefficient.

**averagePathLength()**

Mean shortest-path length (hops, ignoring direction) over all connected pairs.

**diameter()**

The longest shortest path (in hops, ignoring direction) between any two connected nodes.

**egoNetwork(center, depth=1)**

egoNetwork(node, depth=1): the node and everything within `depth` hops of it (ignoring direction).

**giantComponent()**

The nodes of the largest connected component (ignoring direction).

**graphStats()**

Print a summary: size, density, degree, components, diameter, average path length, clustering, transitivity, degree assortativity, reciprocity (directed edges) and (if computed) modularity.

**groupsOfSize(field, smallest, largest=None)**

groupsOfSize(field, min[, max]): the nodes (or edges) whose value of field is shared by at least min (and at most max) elements — e.g. drop tiny communities with showOnly(groupsOfSize(modularity_class, 5)).

**heatMap(center, near='red', far='lightgray', unreachable='white')**

heatMap(node) colours every node by its distance (hops, ignoring direction) from node — near colour at the node, fading to far — and stores the distance in the hops field (-1 if unreachable). Gephi's heat map tool.

**interEdges(field)**

interEdges(field): edges joining nodes with different values of the field.

**intraEdges(field)**

intraEdges(field): edges whose two endpoints share the field's value (e.g. edges inside each community).

**kCore(k)**

kCore(k): the nodes of the k-core — the largest subgraph in which every node has at least k neighbours. E.g. showOnly(kCore(3)).

**louvainClusters(resolution=1.0, seed=0)**

louvainClusters() finds communities by Louvain modularity optimisation (Gephi's Modularity statistic) using edge weights, stores each node's community in the modularity_class field (0 = largest) and returns the communities. Try colorize(modularity_class). resolution > 1 gives more, smaller communities; seed=None varies the result between runs.

**modularity(field)**

modularity(field): the modularity Q of the partition given by a node field (e.g. modularity(dept), modularity(modularity_class)).

**mutualEdges()**

Directed edges whose reverse edge also exists.

**paint(center, color, depth=0)**

paint(node, color, depth=0) colours a node, and with depth=k everything within k hops of it (Gephi's brush).

**radius()**

The smallest eccentricity among nodes with at least one neighbour.

**recompute(*fields)**

recompute(pagerank, coreness, …) recalculates computed fields (betweenness, pagerank, closeness, …) after the graph has changed; recompute() redoes every computed field that has been used so far.

**shortestPath(source, target, weighted=False)**

shortestPath(a, b): the nodes and edges of a shortest path from a to b (following edge direction), or an empty set if there is none. weighted=true uses edge weights as lengths. E.g. shortestPath(a, b).color = red

**showAll()**

Make every node and edge visible again.

**showOnly(seq)**

showOnly(seq) hides every node and edge not in seq (nodes in seq keep the edges between them; edges in seq keep their endpoints). Undo with showAll().

## Clustering and generators

**biComponentClusters()**

Biconnected components (ignoring edge direction).

**edgeBetweennessClusters(count)**

Split the graph into `count` communities with Girvan-Newman.

**makeBarabasiAlbertRandom(*args)**

makeBarabasiAlbertRandom(vert, edges, evolve): preferential attachment; starts from `vert` nodes and adds `evolve` nodes with `edges` edges each.

**makeEppsteinRandom(*args)**

makeEppsteinRandom(nodes, edges, r): power-law graph by r rounds of degree-biased rewiring of a uniform random graph (Eppstein & Wang).

**makeErdosRenyiRandom(*args)**

makeErdosRenyiRandom(nodes, p): each edge present with probability p.

**makeKleinbergRandom(*args)**

makeKleinbergRandom(size, clust): Kleinberg small world on a size x size torus with one long-range edge per node, distance exponent clust.

**makeLattice1DRandom(*args)**

makeLattice1DRandom(nodes, toroidal): a path (or ring if toroidal).

**makeLattice2DRandom(*args)**

makeLattice2DRandom(size, toroidal): a size x size grid (torus if toroidal).

**makeSimpleDirectedRandom(*args)**

makeSimpleDirectedRandom(nodes, edges): uniform random directed graph.

**makeSimpleRandom(*args)**

makeSimpleRandom(nodes, edges): uniform random undirected graph.

**weakComponentClusters()**

Connected components (ignoring edge direction).

## Structure: bridges, cut nodes, backbones, directed graphs, mixing, comparing partitions

**ancestors(n)**

ancestors(n): every node with a directed path to n (upstream: what n depends on or cites, transitively). Undirected edges count both ways.

**assortativity(field=None)**

assortativity() is the degree assortativity of the graph: positive when hubs link to hubs, negative when hubs link to small nodes. assortativity(dept) measures homophily by a field: near 1 = ties stay within the same value, 0 = random mixing, negative = ties cross values. Numeric fields use the correlation of values across edges.

**bridges()**

bridges(): the edges whose removal would split a connected part of the graph in two (direction ignored; a pair joined by parallel edges is never a bridge). bridges().color = red shows the weak ties.

**compareClusters(field1, field2, method='nmi')**

compareClusters(modularity_class, dept) says how well two partitions of the nodes agree, over the nodes that have both values: method “nmi” (normalized mutual information, 0..1), “ari” (adjusted Rand: 0 = chance, 1 = identical), “rand” or “vi” (variation of information: 0 = identical). Works for edge fields too.

**cutNodes()**

cutNodes(): the nodes whose removal would split a connected part of the graph (articulation points; direction ignored).

**descendants(n)**

descendants(n): every node reachable from n along directed edges (downstream: what depends on n, or what n leads to).

**feedbackEdges()**

feedbackEdges(): directed edges that close cycles (back edges of a depth-first search, plus self-loops); removing or reversing them leaves the directed part acyclic. A small set, not necessarily the smallest.

**mixingMatrix(field, show=True)**

mixingMatrix(dept) counts the edges between each pair of values of a node field (dept1-dept1, dept1-dept2, ...), printed as a table and returned as {value: {value: count}}. Direction is ignored. With many values only the 20 most common are printed.

**reciprocity()**

reciprocity(): the share of directed edges whose reverse edge also exists (1 = every tie returned). Undirected edges are left out.

**spanningTree(maximum=False, weighted=True)**

spanningTree() returns the edges of a minimum spanning tree (a forest if the graph is in pieces) by edge weight: the cheapest backbone that keeps everything connected. spanningTree(maximum=True) keeps the strongest ties instead, a good backbone when weight means strength: showOnly(spanningTree(maximum=True)). weighted=False ignores weights.

**transitiveReduction()**

transitiveReduction(): in a directed acyclic graph, the edges implied by longer paths (a -> c when a -> b -> c exists, and repeated edges). They are marked in the edge field `redundant` and returned, so transitiveReduction().visible = false shows the backbone of a citation or dependency graph. Only directed edges count; cycles are an error (see feedbackEdges).

**transitivity()**

transitivity(): the share of connected triples that close into triangles (global clustering coefficient; direction ignored).

**triadCensus(show=True)**

triadCensus() counts the 16 kinds of directed triads (Holland and Leinhardt): how often three nodes form a cycle, a transitive triple, a star and so on. Undirected edges count as mutual. Prints the types that occur and returns {type: count}.

## networkx bridge and sample graphs

**`fromNetworkX(G, name=None)`**

fromNetworkX(G) replaces the graph with networkx graph G: its node and edge attributes become fields (types from the values), `pos` or x/y give positions (otherwise a spring layout), and direction is kept. String keys become node names; other keys become n0, n1, … with the original in `nxkey`. fromNetworkX(G, "name") opens it in a new tab instead and switches to it (use("main") goes back).

**`makeSample(name=None)`**

makeSample("karate") loads a classic network: karate, lesmis, florentine, davis, petersen, kite, tutte, dodecahedron, caveman. makeSample() lists them. They come with their attributes (karate's `club`, lesmis' `weight`, davis' `bipartite`) and a spring layout.

**`setField(values, name, kind=None)`**

setField(values, "name") stores a result as a field: a dict from nodes (or node names) to values, as networkx centralities return; a dict from node pairs (a, b) to values gives an edge field; a list of node sets (as community algorithms return) gives each node its set's index, largest first. The type is taken from the values. Returns the field. setField(nx.core_number(toNetworkX()), "core") setField(nx.community.greedy_modularity_communities(toNetworkX()), "greedy")

**`toNetworkX(directed=None, visibleOnly=False)`**

toNetworkX() returns the graph as a networkx graph, to use any networkx algorithm: node keys are node names, and every field (dept, color, x, y, pagerank once used, …) is an attribute; positions are also `pos`. directed=True/False forces the type (a mixed graph becomes directed, with undirected edges both ways); visibleOnly=True leaves out hidden elements. Bring results back with setField: import networkx as nx setField(nx.katz_centrality(toNetworkX()), "katz")

## igraph: communities, large-graph layouts, similarity, patterns, cuts and flows

**`bipartiteLayout(typeField, hgap=1, vgap=1)`**

bipartiteLayout(field) draws a two-mode network in two rows (by a field with two values, e.g. bipartite in makeSample("davis")), ordering each row to reduce edge crossings.

**`cohesiveBlocks()`**

cohesiveBlocks() finds nested groups that stay connected after removing any k-1 nodes (Moody and White's cohesive blocking): a structural view that k-cores miss. Each node's highest cohesion is stored in `cohesion`; the blocks are returned, most cohesive last. Slow beyond a few thousand nodes.

**`cutDendrogram(count)`**

cutDendrogram(n) re-cuts the hierarchy of the last walktrapClusters or fastGreedyClusters into n communities (same field), e.g. driven by a Slider to explore coarser and finer groupings.

**dhLayout(seed=0)**

dhLayout() is the Davidson-Harel layout: simulated annealing that spreads nodes evenly and avoids edge crossings and nodes on edges. Slow but tidy: for figures of up to a few hundred nodes.

**dominatorTree(root)**

dominatorTree(root) follows directed edges from root and finds, for every node it reaches, its immediate dominator: the last node every path from root must pass through (who controls access to whom). Stored in `idom` (a node name); returns the nodes root reaches.

**drlLayout(preset='default', seed=0)**

drlLayout() is a force-directed layout for large graphs (DrL / OpenOrd): tens of thousands of nodes in seconds, with clusters pulled well apart. preset: “default”, “coarsen” (more separated clusters), “coarsest”, “refine” (polish an existing layout) or “final”.

**eigenvectorClusters(count=None)**

eigenvectorClusters() splits the graph by the leading eigenvector of the modularity matrix (Newman), repeatedly; count limits the number of communities. Stored in eigen_class.

**fastGreedyClusters(count=None)**

fastGreedyClusters() merges communities greedily by modularity (Clauset-Newman-Moore), building a hierarchy; count as in walktrapClusters. Stored in fastgreedy_class; cutDendrogram(n) recuts.

**findPattern(pattern, induced=False, limit=100)**

findPattern(pattern) finds every place where a small pattern occurs in the graph (subgraph isomorphism, LAD algorithm). The pattern is a list of pairs, e.g. findPattern([(“a”,“b”), (“b”,“c”), (“c”,“a”)]) for triangles, or a networkx graph (nx.cycle_graph(4), nx.star_graph(3)). induced=True requires no extra edges among the matched nodes. Returns up to `limit` matches, each a set of nodes; the same nodes matched in another order count once.

**fluidClusters(k, seed=0)**

fluidClusters(k) splits a connected graph into exactly k communities (fluid communities). On a graph in pieces run it on one part, e.g. in subgraph(giantComponent()). Stored in fluid_class.

**infomapClusters(trials=10, seed=0)**

infomapClusters() finds communities by Infomap (how a random walker’s flow stays inside groups), following edge direction and weights; a good choice for directed networks such as citations or web links. Stored in infomap_class.

**labelPropagationClusters(seedField=None, seed=0)**

labelPropagationClusters() finds communities very fast (linear time) by letting each node adopt its neighbours' most common label; good for very large graphs, rougher than Leiden. labelPropagationClusters(field) starts from the values of a field and keeps those nodes fixed, filling in the rest (semi-supervised). Stored in lpa_class.

**largestCliques()**

largestCliques() returns the largest cliques (groups where everyone is linked to everyone), each a set of nodes. Exponential in the worst case, fast on sparse graphs.

**leidenClusters(resolution=1.0, objective='modularity', iterations=-1, seed=0)**

leidenClusters() finds communities with the Leiden algorithm (better than Louvain: every community is internally connected) and stores them in leiden_class (0 = largest). resolution > 1 gives more, smaller communities. objective="CPM" (constant Potts model) avoids modularity's resolution limit; its resolution is a density, typically 0.001-0.1. iterations=-1 runs until stable; seed=None varies the result.

**lglLayout(root=None, seed=0)**

lglLayout() is the Large Graph Layout: grows the drawing outward from a root node (default: a random one, or lglLayout(v1)), suited to big, tree-like networks.

**maxFlow(a, b, capacity='weight')**

maxFlow(a, b) computes the maximum flow from a to b with edge weights as capacities (capacity=None: 1 per edge), following edge direction when the graph has directed edges. Each edge's flow is stored in the edge field `flow`; returns the total.

**maxMatching(typeField=None)**

maxMatching() returns a largest set of edges with no node in two of them (pairs everyone it can); maxMatching(field) matches across the two values of a field in a two-mode network, using weights.

**minCut(a, b, capacity=None)**

minCut(a, b) returns the fewest edges whose removal separates a from b (direction ignored); minCut(a, b, weight) the cut of least total weight. Prints the cut's size.

**optimalClusters()**

optimalClusters() finds the partition with the highest possible modularity (exact, by integer programming): only for small graphs (up to about 100 nodes), as a baseline for the other methods. Stored in optimal_class.

**personalizedPagerank(seq, damping=0.85)**

personalizedPagerank(seq) ranks every node by how easily a random walk restarting at the given nodes reaches it: relevance to a chosen set (e.g. personalizedPagerank(sel) or personalizedPagerank(v1)). Stored in the field `ppr`.

**predictLinks(top=20, method='adamic_adar', show=True)**

predictLinks() lists likely missing edges: pairs of nodes that aren't linked but share neighbours, scored by "adamic_adar" (shared neighbours, rare ones counting more), "jaccard", "common" (count of shared neighbours) or "resource" (resource allocation). Returns the `top` pairs as (node, node, score), best first.

**rewire(times=None, name=None, seed=0)**

rewire() makes a randomised copy of the graph in a new tab keeping every node's degree (repeated swaps of edge ends, 10 x edges by default): the null model for "is this structure more than chance?". Compare e.g. transitivity() or modularity here and there. Edge weights travel with the swapped ends.

**similarTo(n, method='jaccard', top=10)**

similarTo(v1) scores every node by how much its neighbourhood overlaps v1's, stores the score in the field `similarity` and returns the `top` most similar nodes. method: "jaccard" (shared neighbours / all neighbours of either), "dice" (2 x shared / sum of degrees), or "adamic_adar" (shared neighbours, rare ones counting more). Direction is ignored.

**simplify(combine='sum', loops=True)**

simplify() merges parallel edges between the same two nodes (and the same direction) into one, whose weight combines theirs: combine="sum" (default), "max", "min", "mean" or "first"; loops=True also removes self-loops. One undo step.

**spinglassClusters(spins=25, gamma=1.0, seed=0)**

spinglassClusters() finds communities by simulated annealing of a spin model (Reichardt-Bornholdt); handles negative edge weights (signed networks). Slow: for graphs up to about 2,000 nodes. spins is the most communities allowed, gamma > 1 favours smaller ones. Stored in spinglass_class. Needs a connected graph.

**umapLayout(minDist=0.01, epochs=500, seed=0)**

umapLayout() places nodes by UMAP on the graph's connections: tight, well-separated islands for clusters. Edge weights count as similarity. minDist (0..1) is how tightly points may pack; more epochs, more polish.

**useIgraph(on=True)**

useIgraph(false) makes the statistics use networkx / numpy instead of igraph (to compare results or work around a problem); useIgraph() turns igraph back on. Prints which engine is active.

**voronoiClusters()**

voronoiClusters() picks central "generator" nodes automatically and gives every other node to its nearest one (by weighted distance); the communities are stored in voronoi_class and the generators returned.

**walktrapClusters(count=None, steps=4)**

walktrapClusters() groups nodes that short random walks (steps long) tend to stay among, building a hierarchy; count=None cuts it where modularity is highest, walktrapClusters(8) gives 8 communities. Stored in walktrap_class; cutDendrogram(n) re-cuts it.

## Layouts

**binPackLayout(rescale=True)**

binPackLayout() packs the connected components next to each other, first rescaling them to similar densities; binPackLayout(false) packs them without rescaling.

**centerAfterLayout(state)**

centerAfterLayout(state): whether the view re-centres after each layout (default true). Disable it to control the view yourself.

**circleLayout(field=None, center=None, x=None, y=None)**

circleLayout() places all nodes evenly on a circle (radius 450 around 500,500).

circleLayout(field, centerNode) puts centerNode at (500, 500) and its neighbours on a circle around it, ordered by `field`, at a radius proportional to `field` (a node field of the neighbours, or an edge field of the connecting edges). Other nodes are left alone. circleLayout(field, centerNode, x, y) uses (x, y) as the centre.

**dagLayout()**

dagLayout() lays out a directed acyclic graph with a spring layout, keeping each node on a horizontal line for its level (sources at the top), for 500 iterations (fewer on graphs over 1000 nodes). Cycles are broken first.

**forceAtlas2Layout(iterations=None, scaling=None, gravity=1.0, strongGravity=False, linLog=False, dissuadeHubs=False, preventOverlap=False, edgeWeightInfluence=1.0)**

forceAtlas2Layout() runs Gephi's ForceAtlas2 from the current positions (animated; Stop ends it early). Options, by keyword: iterations steps to run (default 500, fewer on big graphs) scaling repulsion strength (default 2, or 10 under 100 nodes) gravity pull toward the centre (default 1) strongGravity=true gravity grows with distance (keeps components close) linLog=true LinLog mode: tighter, more separated clusters dissuadeHubs=true pushes hubs to the periphery (authorities central) preventOverlap=true keep node discs apart (best run after a first pass) edgeWeightInfluence

0 ignores weights, 1 uses them, >1 exaggerates E.g. forceAtlas2Layout(linLog=true, preventOverlap=true)

**`frLayout(width=None, height=None)`**

frLayout() runs Fruchterman-Rheingold (as in Pajek) starting from the current positions and fits the result into a 1000 x 1000 grid with cells the size of the largest node; frLayout(width, height) uses that area.

**`gemLayout(seed=None)`**

gemLayout() runs the GEM force-directed layout (Frick et al.); gemLayout(seed) seeds the random generator for a repeatable layout.

**`hiveLayout(axis=None, position=None, scale='rank', axes=None, inner=60, length=440, curved=True)`**

hiveLayout(axis, position) draws a hive plot (Krzywinski et al.): one radial axis per value of the node field `axis` (the most common values, at most `axes` of them, the rest merged into "other"), each node placed along its axis by the numeric field `position` (default totaldegree), and edges drawn as curves between axes. scale is "rank" (default; evenly spread), "linear" or "log". Without `axis`: for directed graphs the classic source / hub / sink axes (only outgoing, both, only incoming edges), otherwise leaves / connectors / hubs by degree. The axes are drawn with their labels; any later layout removes them. E.g. hiveLayout(dept, pagerank) or hiveLayout(modularity_class, axes=5)

**`isomLayout(max=None)`**

isomLayout() runs the JUNG ISOM (self-organising map) layout for 2000 epochs; isomLayout(n) runs n epochs.

**`jfrLayout()`**

jfrLayout() runs the JUNG Fruchterman-Rheingold layout (random start in a 1000 x 1000 area, up to 700 cooling iterations).

**`jkkLayout1()`**

jkkLayout1() runs the JUNG Kamada-Kawai layout: unweighted graph distances, random start, 2000 iterations each moving the worst-placed node.

**`jkkLayout2()`**

jkkLayout2() runs the JUNG Kamada-Kawai layout with integer distances (same algorithm as jkkLayout1).

**jSpringLayout(max=None)**

jSpringLayout() runs the JUNG spring layout from random positions until it settles or for 500 iterations (fewer on graphs over 1000 nodes); jSpringLayout(n) runs exactly n iterations.

**kkLayout(width=None, height=None)**

kkLayout() runs Kamada-Kawai on each connected component, starting from a circle, then fits the result into 1000 x 1000; kkLayout(width, height) uses that area instead.

**labelAdjustLayout(margin=2)**

labelAdjustLayout() moves nodes just enough that nodes and their visible labels stop overlapping (Gephi's Label Adjust).

**mdsLayout()**

mdsLayout() places nodes by classical multidimensional scaling of the edge weights (an edge's weight is the distance between its endpoints, every other pair is at distance 1). Nodes without edges are not moved.

**mirrorLayout(horizontal=True)**

mirrorLayout() flips the layout left-right; mirrorLayout(false) flips it upside down.

**moveLayout(newX, newY)**

moveLayout(x, y) moves the layout so its top-left corner is at (x, y).

**noverlapLayout(margin=5, ratio=1.2)**

noverlapLayout() nudges nodes apart until none overlap (Gephi's Noverlap): each node is treated as a disc of ratio x its size plus margin.

**physicsLayout(max=None)**

physicsLayout() runs the graphopt physics layout (charged nodes, springs) for 500 steps from the current positions (fewer on graphs over 1000 nodes); physicsLayout(n) runs n steps.

**radialLayout(center, edges=None)**

radialLayout(centerNode) lays the graph out as a radial tree around centerNode (placed at 0,0), following a breadth-first spanning tree (Eades' radial drawing). Nodes not reachable from the centre stay put. radialLayout(centerNode, edges) restricts the tree to the given edges.

**randomLayout(width=None, height=None)**

randomLayout() places nodes at random in a 1000 x 1000 area; randomLayout(width, height) uses a width x height area.

**rescaleLayout(*d)**

rescaleLayout(percent) scales the layout by percent (0.5 = half size) about its top-left corner; rescaleLayout(xpercent, ypercent) scales each axis separately; rescaleLayout(width, height) with integers fits the layout into a width x height box at the origin.

**ringMove(node, index)**

ringMove(node, k) moves a node to position k around the circle made by circleLayout() and respaces the ring (what dragging a node around the circle does).

**rotateLayout(degrees=90)**

rotateLayout(degrees) turns the layout clockwise about its centre.

**scatterLayout(xfield, yfield, width=1000, height=1000)**

scatterLayout(xfield, yfield) places nodes by two numeric fields (x to the right, y upwards) in a width x height area. Dragging a node then edits those two values.

**springLayout(max=None)**

springLayout() runs the GUESS spring layout from the current positions until it settles or for 1000 iterations (fewer on graphs over 1000 nodes: 1000 * 1000 / nodes, at least 100); springLayout(n) runs exactly n iterations. Stop it early with the Stop button.

**sugiyamaLayout(bends=False)**

sugiyamaLayout() draws the graph in layers (Sugiyama): edges are made acyclic, nodes put on levels so every edge points down, crossings reduced by barycentres, on a 250 x 150 grid. sugiyamaLayout(true) also routes long edges through dummy positions.

**sugiyamaLayout2()**

sugiyamaLayout2() is a simpler layered layout: levels 200 apart, each node given horizontal room in proportion to its children.

**translateLayout(newX, newY)**

translateLayout(dx, dy) moves every node by (dx, dy).

**treeLayout(root=None, horizontal=False, spacing=None)**

treeLayout() draws each connected component as a tidy tree (a breadth-first spanning tree from its best-connected node, or from root), parents centred over their children; horizontal=true grows it left to right; spacing sets the distance between neighbouring leaves.

**weightedKKLayout()**

weightedKKLayout() runs the JUNG Kamada-Kawai layout using shortest paths where each edge's `weight` is its length.

**weightedSpringLayout(minLength, maxLength, max=None)**

weightedSpringLayout(minLength, maxLength[, iterations]) is springLayout with each edge's natural length mapped linearly from its `weight` (lowest weight -> minLength, highest -> maxLength). Iterations as for springLayout.

## Display: hulls, legends, background, edges, labels, selection, export

**createConvexHull(seq, color='200,200,255,90')**

Shade the convex hull around the nodes in seq (edges contribute their endpoints).

**curvedEdges(on=True)**

curvedEdges() draws every edge as a circular arc running clockwise from source to target, as in Gephi's preview, so a->b and b->a separate and direction reads without arrows. curvedEdges(false) goes back to straight lines; curvedEdges(k) sets the curvature: the arc radius is edge length / k (default 1.2, as in Gephi; 2 gives semicircles).

**exportCGM(filename=None)**

Not available in the browser (the Java version wrote CGM); use exportSVG or exportPDF.

**exportEMF(filename=None)**

Not available in the browser (the Java version wrote EMF); use exportSVG or exportPDF.

**exportEPS(filename='guess.eps')**

Save the display as EPS (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

**exportGIF(filename='guess.gif')**

Save the display as GIF (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

**exportJAVA(filename=None)**

Not available in the browser (the Java version wrote JAVA); use exportSVG or exportPDF.

**exportJPG(filename='guess.jpg')**

Save the display as JPG (downloaded by the browser).

**exportPDF(filename='guess.pdf')**

Save the display as PDF (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

**exportPNG(filename='guess.png')**

Save the display as PNG (downloaded by the browser).

**exportPS(filename='guess.ps')**

Save the display as PS (downloaded by the browser): an image of the current view inside the file. For vector output use exportSVG.

**exportSVG(filename='guess.svg')**

Save the display as SVG (downloaded by the browser). Vector: the whole graph, not just the visible area.

**exportSWF(filename=None)**

Not available in the browser (the Java version wrote SWF); use exportSVG or exportPDF.

**getConvexHulls()**

All hulls currently shown.

**getDisplayBackground()**

The display background color, or None when it follows the light/dark theme.

**GradientLegend(start, end, lo=0, hi=1, ticks=None)**

A color ramp legend: GradientLegend(startColor, endColor, min, max[, tickEvery]) shows the ramp from min to max, with a tick mark every tickEvery units.

**hideOverlappingLabels(on=True)**

hideOverlappingLabels() hides labels that would overlap a label of a bigger node, rechecked as you zoom; hideOverlappingLabels(false) shows all.

**labelOutlines(on=True)**

labelOutlines() draws a halo in the background colour around labels so they stay readable over edges; labelOutlines(false) turns it off.

**Legend()**

A legend box on the display. `l = Legend(); l.add(bob, "managers")`.

Entries snapshot the look of the node, edge or hull at the time they are added.

**`overrideArrowLength(length)`**

Fix the arrowhead length (graph units) for all directed edges; None resets.

**`overrideArrowWidth(width)`**

Fix the arrowhead width (graph units) for all directed edges; None resets.

**`removeBackgroundImage()`**

Remove the background image set with setBackgroundImage.

**`removeConvexHull(hull)`**

Remove one hull.

**`removeConvexHulls(seq=None)`**

Remove the given hulls, or all hulls.

**`select(seq=None)`**

select(seq) selects these nodes on the display (edges select their endpoints) and binds them to `sel`, as a box, lasso or shift-click does; select() clears the selection. On the display: shift-drag draws a box, alt-drag a lasso, shift-click adds or removes one node.

**`setBackgroundImage(source, x=0, y=0)`**

Show an image (a file you opened, or a URL) behind the graph with its top-left corner at (x, y) in graph coordinates.

**`setDisplayBackground(color)`**

Set the display's background color; setDisplayBackground(None) goes back to the theme's (white, or dark in dark mode).

**`setEdgeColorMode(mode='original')`**

setEdgeColorMode("mixed") draws each edge in the blend of its endpoints' colours; "source" / "target" use one endpoint's colour; "original" (the default) uses the edge's own color field. As in Gephi's preview.

**`startMovie(fps=30, filename='guess.webm')`**

Record the display (graph canvas) as a WebM video until stopMovie(). Everything shown — layouts, morphs, edits, panning — is captured.

**`stopMovie()`**

Stop recording and download the video.

## Styles and visual mappings

**`applyStyle(name)`**

applyStyle(name) runs a style's commands (one undo step).

**`defineStyle(name, commands)`**

defineStyle(name, [command, …]) saves a named style: Gython lines that set colours, sizes, labels, legends or display settings. Styles are saved with the session; the style panel builds them for you.

**`getStyles()`**

The saved styles: {name: [commands]}.

**`labelBy(field, show=True)`**

labelBy(field) sets each node's (or edge's) label to its value of field and shows the labels; labelBy(field, false) sets them without showing.

**`legendFor(field, title=None)`**

legendFor(field) adds a legend explaining the current colours by field: one entry per value (for categories, as after colorize), or a gradient from the lowest to the highest value (for numbers, as after colorBy). The legend follows the colours: run colorize or colorBy again and it updates (until you add or remove entries yourself).

**`removeLegend(legend)`**

removeLegend(lg) or removeLegend(2) removes one legend (the number is the one shown as <legend 2: …>; the × on a legend runs this).

**`removeLegends()`**

removeLegends() removes every legend.

**`removeStyle(name)`**

removeStyle(name) deletes a saved style.

**`shapeBy(field)`**

shapeBy(field) gives each value of a node field its own shape (circle, square, rounded square, repeating), the most common value first.

**`table(seq=None)`**

table() opens the data table of all nodes; table(seq) shows just those nodes or edges (e.g. table(sel), table(freq > 10), table(g.edges)).

## Pie nodes, borders and edge bundling

**`borderBy(fld, width=2.5)`**

borderBy(field) gives node borders a distinct colour per value of a field (as colorize does for fills) and widens them.

**`bundleEdges(fld, strength=0.85)`**

bundleEdges(field) routes each edge between two groups (values of a node field, e.g. modularity_class) through the groups' centres, so edges between the same groups merge into bundles. strength 0..1 (0 = straight). Layouts and unbundleEdges() straighten them again.

**`pieChart(fields, palette=None)`**

pieChart([f1, f2, …]) draws each node as a pie whose slices are the node's values of those numeric fields; palette=colors sets slice colours.

**`pieNeighbors(fld)`**

pieNeighbors(field) draws each node as a pie of its neighbours' values of a categorical field (the most common values; the rest as "other").

**`pieOff()`**

Draw nodes normally again (after pieChart / pieNeighbors).

**`unbundleEdges()`**

Draw edges straight again after bundleEdges.

## Grouping: collapse and expand

**`collapse(seq, name=None)`**

collapse(seq) replaces those nodes with one meta-node (see module help); collapse(seq, "name") names it. Returns the meta-node.

**`collapseBy(field, minimum=2)`**

collapseBy(field) collapses each group of nodes sharing a value of field (with at least `minimum` members) into a meta-node named after the value.

**`expand(meta)`**

expand(metaNode) brings back the nodes a meta-node stands for (also: double-click it). Returns them.

**`expandAll()`**

Expand every collapsed group (repeatedly, for nested groups).

**isGroup(node)**

True if node is a collapsed group (meta-node).

## Several graphs: subgraphs and tabs

**closeGraph(name)**

closeGraph(name) discards a graph that isn't the current one.

**graphNames()**

The graphs in this session; the current one first.

**newGraph(name='untitled')**

newGraph(name) starts an empty graph and switches to it.

**renameGraph(name)**

renameGraph(name) renames the current graph.

**subgraph(seq=None, name=None)**

subgraph(seq) makes a new graph from those nodes (edges select their endpoints) and the edges among them, keeping fields, positions and look, and switches to it; subgraph(seq, "name") names it. Use use(name) to go back.

**use(name)**

use(name) switches to another graph of this session (see graphNames()).

## Time: ranges, years and the timeline

**showTime(t, field=None, window=None)**

showTime(t) shows only the nodes and edges whose ranges contain time t (elements without ranges stay visible); showTime(t, field) shows those whose numeric field is <= t (window=w: only within the last w). Edges are shown only when both ends are.

## Presentations (slides)

**addSlide(note='', camera=None, at=None)**

addSlide(note) saves the current graph (as a state) and view as a slide; camera=[x, y, zoom] is filled in by the page; at=k inserts it at k.

**moveSlide(index, to)**

moveSlide(k, j) moves slide k to position j.

**`noteSlide(index, note)`**

noteSlide(k, text) changes slide k's note.

**`removeSlide(index)`**

removeSlide(k) deletes slide k.

**`showSlide(index, duration=1200)`**

showSlide(k) morphs the graph to slide k's state and moves the camera.

**`slides()`**

The slides' notes, in order.

## Data from the web: URLs, SPARQL, NDEx, Neo4j, packages

**`installPackage(name)`**

installPackage("name") installs a Python package into this session (pure-Python wheels from PyPI, or Pyodide's own packages such as scipy), in the background; a note appears when it's ready to import.

**`makeFromCypher(query, url='http://localhost:7474', user=None, password=None, database='neo4j')`**

makeFromCypher(query, url, user, password) runs a Cypher query on a Neo4j server's HTTP API and builds the graph from the nodes and relationships it returns (properties become fields).

**`makeFromNDEx(uuid, server='https://www.ndexbio.org')`**

makeFromNDEx(uuid) opens a public network from the NDEx repository (Cytoscape's network commons), with its attributes and layout.

**`makeFromSPARQL(query, endpoint='https://query.wikidata.org/sparql', directed=True)`**

makeFromSPARQL(query) builds a graph from a SPARQL query whose rows bind ?source and ?target (optionally ?sourceLabel / ?targetLabel, used as labels; any other variables become edge fields). E.g. a Wikidata query for "influenced by" links.

**`makeFromURL(url, format=None)`**

makeFromURL(url) downloads a graph (GDF, GEXF, GraphML, GML, DOT, Pajek, DL, CSV, a .guess session or a .py script) and opens it; format=".gexf" etc. when the URL doesn't end in the extension.

**`sparql(query, endpoint='https://query.wikidata.org/sparql')`**

sparql(query[, endpoint]) runs a SPARQL query (Wikidata by default) and returns the rows as a list of {variable: value} dicts.

## States and morphing

**`getEdgeAppearAfter()`**

The current edgeAppearAfter fraction (see setEdgeAppearAfter).

**`getEdgeDisappearBy()`**

The current edgeDisappearBy fraction (see setEdgeDisappearBy).

**`getNodeAppearAfter()`**

The current nodeAppearAfter fraction (see setNodeAppearAfter).

**`getNodeDisappearBy()`**

The current nodeDisappearBy fraction (see setNodeDisappearBy).

**`getStates()`**

Names of the saved states.

**`ls(state)`**

Load a saved state: restores which nodes/edges exist and all their values.

**`morph(state, duration=1000)`**

Animate from the current display to a saved state over `duration` ms.

**`setEdgeAppearAfter(v)`**

Fraction (0..1) of a morph by which edgeAppearAfter takes effect.

**`setEdgeDisappearBy(v)`**

Fraction (0..1) of a morph by which edgeDisappearBy takes effect.

**`setNodeAppearAfter(v)`**

Fraction (0..1) of a morph by which nodeAppearAfter takes effect.

**`setNodeDisappearBy(v)`**

Fraction (0..1) of a morph by which nodeDisappearBy takes effect.

**`ss(state)`**

Save the current graph (membership and all field values) as a named state.

## Ranges (existence intervals)

**`addRange(obj, spec)`**

Attach an existence range like “1,5-10,20” to nodes/edges (adds to any existing range).

**`getRange(obj)`**

The range of a node or edge as a string.

**`searchContained(spec)`**

Nodes and edges whose range lies inside the given range.

**`searchContains(spec)`**

Nodes and edges whose range contains the given range.

**`searchExact(spec)`**

Nodes and edges whose range equals the given range.

**`searchOverlap(spec)`**

Nodes and edges whose range overlaps the given range.

## Charts

**`plotDistrib(field, sortField=None)`**

Each element’s value of field, in rank order (or ordered by sortField).

**`plotHistogram(field, bins=10)`**

Histogram of a numeric field’s values in `bins` equal-width bins.

**`plotSizes(field)`**

Bar chart of how many elements share each value of field (largest first).

**`plotSizesPie(field)`**

Pie chart of how many elements share each value of field.

## Interface building (panels, widgets, timers, events)

**`after(seconds, fn)`**

Call fn() once after `seconds` seconds; returns a timer with .stop().

**Button(*a, **k)**

Button(text, onClick=fn): calls fn() when clicked; .enabled turns it on/off.

**Checkbox(*a, **k)**

Checkbox(text, checked=False, onChange=fn): calls fn(checked) when toggled.

**Dropdown(*a, **k)**

Dropdown(options, value=None, label="", onChange=fn): a choice list; calls fn(value) when the selection changes.

**every(seconds, fn)**

Call fn() every `seconds` seconds; returns a timer with .stop().

**Label(*a, **k)**

Label(text): a line of text in a panel; set .text to change it.

**openURL(url)**

Open a web page in a new browser tab.

**Panel(*a, **k)**

Panel(title, *widgets, position="bottom", vertical=False). Show with ui.dock(p).

**Slider(*a, **k)**

Slider(min, max, value=None, step=1, label="", onChange=None, onSlide=None). onChange fires on release, onSlide while dragging.

**stopTimers()**

Stop all timers.

**TextField(*a, **k)**

TextField(value="", placeholder="", onEnter=None): onEnter(text) on Return.

## Sessions and apps

**exportApp(filename='app.guess', script=None)**

Save a session that opens as an app: with ?app=1 the console is hidden and `script` (Gython source, or the name of a .py file) runs on load — e.g. to dock panels and set graphevents handlers.

**makeFromSession(filename)**

Load a .guess session file, replacing the current graph.

**saveSession(filename='session.guess')**

Save the graph, fields, states, hulls, legends and view to a .guess file (downloaded by the browser). User variables are not saved.

## SQL

**sql(query)**

Two tables are materialised in an in-memory SQLite database on each call:

```
nodes(name, <every node field>)
edges(__edgeid, node1, node2, directed, <every edge field>)

sql("SELECT dept, count(*) FROM nodes GROUP BY dept ORDER BY 2 DESC")
sql("SELECT name FROM nodes WHERE x > y")            -> a set of nodes
sql("SELECT __edgeid FROM edges WHERE freq > 100")   -> a set of edges
sql("UPDATE nodes SET color = 'red' WHERE dept = 'dept3'")
```

SELECTs whose only column is `name` (nodes) or `__edgeid` (edges) return graph elements, so they combine with the rest of Gython; other queries return rows. UPDATEs are written back through the normal field setters (the display updates). Inserting or deleting rows is not supported — use addNode/remove.

## Colors

**averageColor(c1, c2)**

The color halfway between two colors.

**distinctColors(n, seed=1)**

distinctColors(n): n clearly different colours (as "r,g,b" strings), the most different first — the categorical palette colorize(field) uses.

**generateColors(*args)**

generateColors(start, end, n) or generateColors(start, middle, end, n).

**randomColor(alpha=None)**

A random color "r,g,b"; randomColor(alpha) adds an alpha (0..255).

## Examples

The same examples are searchable in the app's Help panel and command palette (Cmd/Ctrl+K).

## Basics

### Colour nodes by a query

```
(freq > 100).color = red
```

### Nodes with a field value

```
dept == 'dept3'
```

### Pattern match names

```
name like 'v1%'
```

### Combine queries

```
(dept == 'dept1') & (totaldegree > 3)
```

### Loop over a set

```
for n in (dept == 'dept3'):
    n.labelvisible = true
```

### Edges between two groups

```
(dept == 'dept1') <-> (dept == 'dept3')
```

### Directed edges

```
v1 -> g.nodes
```

### Any edges touching a node

```
v1 ? g.nodes
```

### Neighbours of a node

```
v1.getNeighbors()
```

### Add nodes and edges

```
addNode('zed', color=red)
addEdge(zed, v1)
```

### Remove and restore

```
remove(totaldegree == 0)
add(g.deleted)
```

### Undo the last command

```
undo()
```

## Appearance

### A colour per category

```
colorize(dept)
```

### Gradient by value

```
colorBy(freq, yellow, red, 'log')
```

**Size by importance**

```
sizeBy(pagerank, 5, 40, 'sqrt')
```

**Size by rank of a field**

```
resizeLinear(totaldegree, 5, 30)
```

**Labels from a field**

```
labelBy(dept)
```

**Shapes per category**

```
shapeBy(dept)
```

**Legend for a mapping**

```
colorize(dept)
legendFor(dept)
```

**Edge width by weight**

```
sizeBy(freq, 1, 8)
```

**Curved, mixed-colour edges**

```
curvedEdges()
setEdgeColorMode('mixed')
```

**Readable labels**

```
labelOutlines()
hideOverlappingLabels()
```

**Shade a group**

```
createConvexHull(dept == 'dept1', '200,200,255,90')
```

**Dark background**

```
setDisplayBackground(black)
```

**Save a named style**

```
defineStyle('by dept', ['colorize(dept)', 'legendFor(dept)'])
applyStyle('by dept')
```

### Layouts

**ForceAtlas2**

```
forceAtlas2Layout()
```

**ForceAtlas2, tighter clusters**

```
forceAtlas2Layout(linLog=true, preventOverlap=true)
```

#### GEM layout

```
gemLayout()
```

#### Kamada-Kawai

```
kkLayout()
```

#### Hierarchy

```
sugiyamaLayout()
```

#### Hive plot

```
hiveLayout(dept, pagerank, axes=5)
```

#### Radial around a node

```
radialLayout(v1)
```

#### Remove overlaps

```
noverlapLayout()
```

#### Pack components

```
binPackLayout()
```

#### Rotate / mirror

```
rotateLayout(90)
mirrorLayout()
```

### Analysis

#### Summary statistics

```
graphStats()
```

#### Communities

```
louvainClusters()
colorize(modularity_class)
```

#### Most central nodes

```
sortBy(pagerank)[-10:]
```

#### Label the hubs

```
(betweenness > 1000).labelvisible = true
```

#### Shortest path

```
shortestPath(v1, v14).color = red
```

#### Distance heat map

```
heatMap(v1)
```

### Groups as charts

```
plotSizesPie(dept)
```

### Distribution chart

```
plotHistogram(pagerank)
```

### SQL over fields

```
sql('select dept, count(*) from nodes group by dept')
```

### Weak points

```
bridges().color = red
cutNodes().size = 30
```

### Backbone

```
showOnly(spanningTree(maximum=True))
```

### Brokers

```
sortBy(constraint)[:5]
```

### Homophily

```
assortativity(dept)
mixingMatrix(dept)
```

### Compare partitions

```
louvainClusters()
compareClusters(modularity_class, dept)
```

### Any networkx algorithm

```
import networkx as nx
setField(nx.community.greedy_modularity_communities(toNetworkX()), 'greedy')
colorize(greedy)
```

### Classic networks

```
makeSample('karate')
colorize(club)
```

### Leiden communities

```
leidenClusters()
colorize(leiden_class)
```

### Communities in directed graphs

```
infomapClusters()
colorize(infomap_class)
```

### Similar nodes

```
similarTo(v1).color = red
```

#### Likely missing links

```
predictLinks(10)
```

#### Find a pattern

```
findPattern([('a','b'), ('b','c'), ('c','a')])
```

#### Large-graph layout

```
drlLayout()
```

### Filtering and selection

#### Show only a core

```
showOnly(kCore(3))
```

#### Ego network

```
showOnly(egoNetwork(v1, 2))
```

#### Largest component

```
showOnly(giantComponent())
```

#### Edges inside communities

```
showOnly(intraEdges(modularity_class))
```

#### Show everything again

```
showAll()
```

#### Select from code

```
select(dept == 'dept2')
```

#### Act on the selection

```
sel.color = orange
center(sel)
```

#### Table of a set

```
table(dept == 'dept3')
```

### Files and sessions

#### Load Gephi's format

```
makeFromGEXF('graph.gexf')
```

#### Load spreadsheets

```
makeFromCSV('edges.csv', 'nodes.csv')
```

#### Export for Gephi

```
exportGEXF('graph.gexf')
```

#### Save a session

```
saveSession('work.guess')
```

#### Save what you ran

```
saveHistory('work.py')
```

#### Snapshot and morph

```
ss('before')
circleLayout()
morph('before', 2000)
```

#### Random graph

```
makeBarabasiAlbertRandom(5, 2, 300)
```

### Interface building

#### A slider that filters

```
s = Slider(freq.min, freq.max, label='min freq')
def update(v):
    (freq < v).visible = false
    (freq >= v).visible = true
s.onChange = update
ui.dock(Panel('Filter', s))
```

#### React to clicks

```
graphevents.clickNode = lambda n: center([n] + n.getNeighbors())
```

#### Record a movie

```
startMovie(30, 'run.webm')
gemLayout()
stopMovie()
```

## Fields

Built-in node fields:

| field | type | default |
|---|---|---|
| `name` | VARCHAR | None (read-only) |
| `x` | DOUBLE | 500.0 |
| `y` | DOUBLE | 500.0 |
| `visible` | BOOLEAN | True |

| field | type | default |
|---|---|---|
| color | VARCHAR | 'cornflowerblue' |
| strokecolor | VARCHAR | None |
| labelcolor | VARCHAR | None |
| fixed | BOOLEAN | False |
| style | INTEGER | 2 |
| width | DOUBLE | 10.0 |
| height | DOUBLE | 10.0 |
| label | VARCHAR | None |
| image | VARCHAR | None |
| labelvisible | BOOLEAN | False |
| indegree | INTEGER | 0 (read-only) |
| outdegree | INTEGER | 0 (read-only) |
| totaldegree | INTEGER | 0 (read-only) |

Built-in edge fields:

| field | type | default |
|---|---|---|
| node1 | VARCHAR | None (read-only) |
| node2 | VARCHAR | None (read-only) |
| directed | BOOLEAN | False (read-only) |
| source | VARCHAR | None (read-only) |
| destination | VARCHAR | None (read-only) |
| color | VARCHAR | 'dandelion' |
| labelcolor | VARCHAR | None |
| visible | BOOLEAN | True |
| width | DOUBLE | 2.0 |
| weight | DOUBLE | 1.0 |
| label | VARCHAR | None |
| labelvisible | BOOLEAN | False |

Computed node fields (calculated the first time they are used, by name or as `n.<field>`; `recompute()` after editing the graph): `pagerank`, `hub`, `authority`, `eigenvector`, `closeness`, `harmonic`, `eccentricity`, `clustering`, `triangles`, `coreness`, `strength`, `component`, `strongcomponent`, `betweenness`, `degrank`, `hits`, `rwbetweenness`, `knn`; edge field `betweenness`.

## Gython syntax

Gython is Python 3 plus these graph operators (see the CHI 2006 paper):

| expression | meaning |
|---|---|
| `freq > 10` | elements whose field matches (==, !=, <, >, <=, >=, like) |
| `(freq > 10).color = blue` | setting a field on a set sets it on every element |
| `a & b`, `a \| b` | intersection / union of element sets |
| `a -> b`, `a <- b` | directed edges from a to b / from b to a |
| `a <-> b`, `a - b` | undirected edges between a and b |
| `a ? b` | any edges between a and b |
| `s.difference(t)` | set difference |

| expression | meaning |
|---|---|
| `bob[1999].color` | a value in a saved state |
| `Node rcontains 5` | range queries (`rcontains`, `roverlaps`, `rcontained`) |
| `g.nodes`, `g.edges` | all nodes / edges |
| `sel` | the current display selection |